系列驱动器

用户手册

V3.5



中文 用户版

刘彦禹 郑淳馨等 编写

2024 年 9 月 25 日

目录 Catalogue

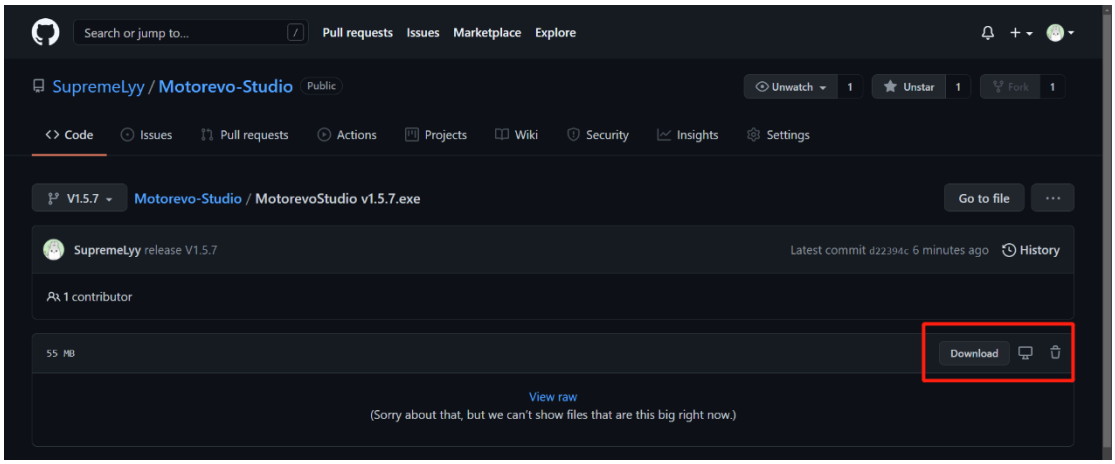
0 声明 3
1 产品简介与外形尺寸 5
1.1 产品简介 5
1.2 接口说明以及电器参数 5
1.2.1 LTH3.2MW-RR60 驱动器（CAN 版本） 6
1.2.2 LTH3.2ML-RR60 驱动器（CAN 版本） 7
3 Safe Torque Off 机制 7
4 开机自检 8
5 校准 10
5.1 编码器校准 10
5.2 电角度校准 10
6 控制与通信 12
6.1 状态定义 12
6.2 控制模式简介 13
6.2.1 伺服模式 Servo Mode 13
6.2.2 力位混合模式 Torque-Position-Mix Control Mode 13
6.2.3 速度模式 Velocity Mode 14
6.2.4 力矩模式 Torque Mode 14
6.2.5 力矩 4 模式 Torque4 Mode 14
6.3 CAN 帧格式定义 15
6.3.1 使用 CAN 控制电机在伺服模式（Servo Mode）运行 15
6.3.2 使用 CAN 控制电机在力位混合模式（P-T-M Mode）运行 17
6.3.3 使用 CAN 控制电机在速度模式（Velocity Mode）运行 19
6.3.4 使用 CAN 控制电机在力矩模式（Torque Mode）运行 20
6.3.5 使用 CAN 控制电机在力矩 4 模式（Torque4 Mode）运行 20
6.5 CAN 通信反馈 21
6.5.1 反馈帧格式 21
6.5.2 反馈帧状态解包 21
6.6 多执行器级联运行 25
6.6.1 CAN 实现多执行器级联运行 25
6.6.2 EtherCAT 实现多执行器级联运行 26
6.7 使用 MParameter Studio 修改电机参数 27
6.7.1 建立连接与保存 27
6.7.2 参数说明 28
6.7.3 修改控制模式 32
6.7.4 固件升级 33
6.8 EtherCAT 控制 34
6.8.1 使用 EtherCAT 控制电机在 CSP 模式运行 34
6.8.2 使用 EtherCAT 控制电机在 CSV 模式运行 37
6.8.3 使用 EtherCAT 控制电机在 CST 模式运行 40
6.9 通过 CAN 更改电机 Flash 中的参数 43
6.9.1 参数读写实例: 44

6.9.2 通过上位机修改 44
7 通过 CAN 总线使用 Motorevo Studio™控制执行器运行实例 46
7.1 Motorevo Studio™界面 46
7.2 通过 Jason 建立 PC-Actuator 连接 48
7.3 使用 Motorevo Studio™驱动电机运行 50
7.4 电流环参数辨识以及电流环带宽测试 52
7.5 位置环、速度环正弦、方波响应测试。 54
8 通过 EtherCAT 使用 TWINCAT3 和控制执行器运行实例 55
8.1 建立 ECAT 连接 55
8.2 导入 XML 文件 59
8.3 在 CSP 模式下运行 61

0 声明

注意，本手册仅适用于 2024 年 9 月 25 日以后出厂的 Motorevo 驱动器（驱动固件号大于 240925）,并需要 Motorevo Studio V3.5 上位机调试。

您可以访问 https://github.com/SupremeLyy/Motorevo-Studio 获取最新的 Motorevo Studio™软件、使用手册以及驱动器固件 。



此外，为了您能尽快上手使用我们的产品，我们强烈推荐您观看以下视频：

【Motorevo 关节驱动器保姆教程】

https://www.bilibili.com/video/BV1Uh4y1C7Xw/?share_source=copy_web&vd_source=0be83eaef43126aba089ca5f7d82e5de

更新：

日期	版本号	内容
2024.1.19	V0.2.1	增加 EtherCAT 部分内容，6.8 章
2024.6.19	V0.2.2	适用于电机固件号为 OXAF0620 以后的电机，修改了开机自检部分，以及故障码 report 部分

2024.7.22	V0.2.3	适用于电机固件号 240722 以后的固件，增加故障码 15，16，增加 USB 可设定参数。
2024.9.25	V3.5	适用于电机固件号 240925 以后的固件，上位机版本大于 V3.5（需要新 Jason 固件），本次更新主要是协议上的更新，变化集中在 6.5,6.7,6.9,7.4 章

1 产品简介与外形尺寸

1.1 产品简介

Motorevo 系列伺服驱动器采用全新自主知识产权的机械结构和减速器设计方案，搭载同类产品中算力最高的驱动控制芯片，基于场定向 Field-Oriented-Control 算法，搭配先进自抗扰控制技术和在线参数辨识技术对速度、角度、力矩进行控制。并支持配合 Motorevo Studio™调参软件进行参数设置并升级固件。

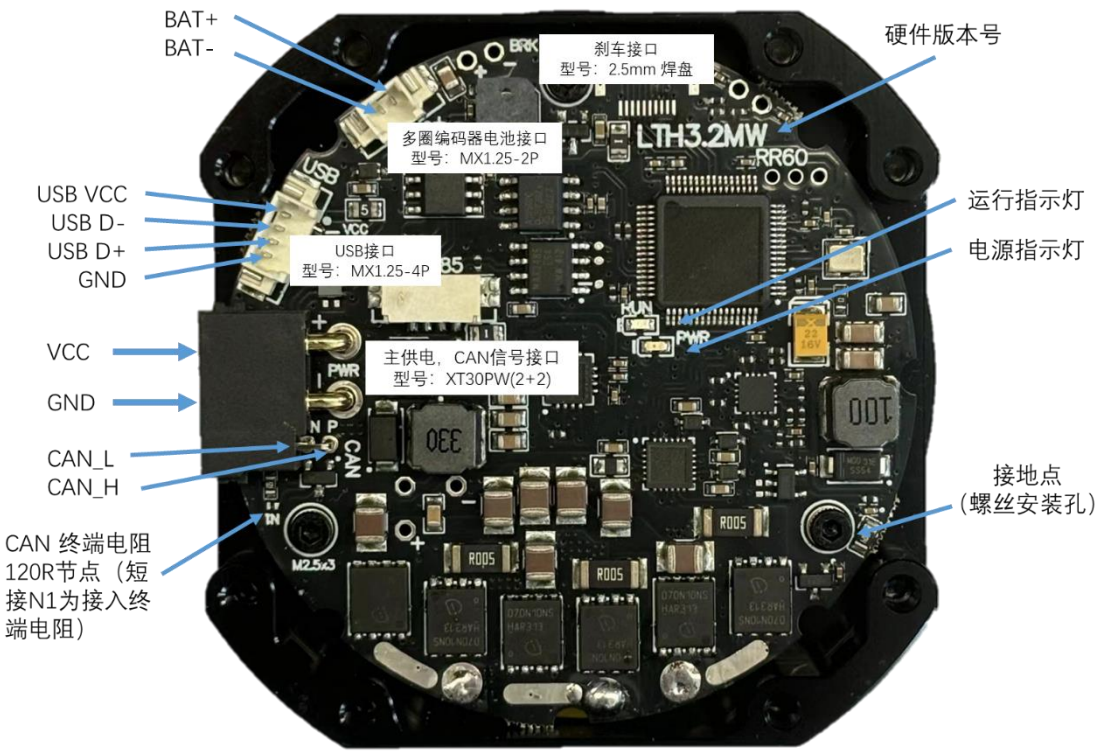
该款驱动器集成有以下功能：

1. 自适应 FOC 正弦调制驱动器
2. 通讯方式为 CAN 或 EtherCAT
3. 支持散热风扇（5v）
4. 支持电磁式刹车（24v 动作/7v 维持）
5. 支持双编码器（输入轴+输出轴）或输入轴多圈绝对值编码器。
6. 支持纯硬件 Safe Torque Off（STO）

1.2 接口说明以及电器参数

Motorevo 驱动器有多种接口形式，请对照您手中的驱动器，自行确定接口。
同时，请您在连接前检查接口有无破损、异物或短路。对于功率接口，尽量避免热插拔，即将插头完整、稳定插入后再上电。对于通信接口，本驱动产品已经具备了一定的抗静电能力，但是仍然请避免人手、金属导体、高压带电导体直接触碰通信接口本体。

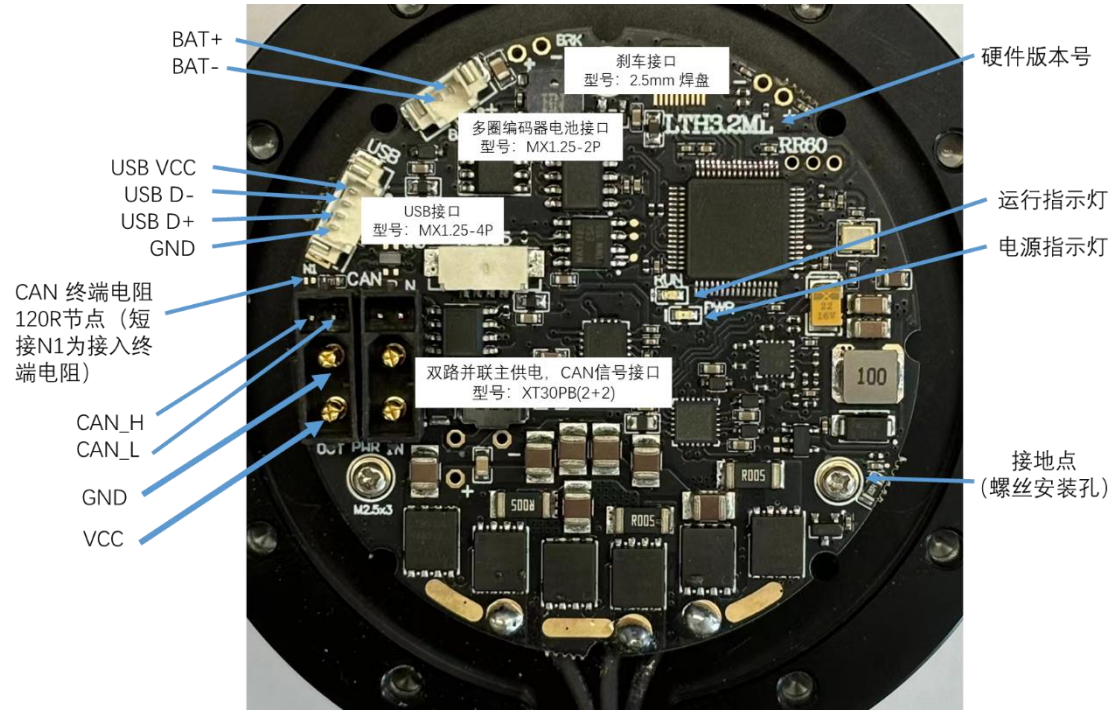
1.2.1 LTH3.2MW-RR60 驱动器（CAN 版本）



电气特性				
参数	**MIN**	**TYP**	**MAX**	**UNIT**
主供电接口 输入电压	12	24/48/72	95	V
主供电接口 输入电流（瞬时）			10	A
主供电接口 输入电流（持续）			4	A
多圈编码器电池供电电压	3.3	3.3	5	V
CAN 总线比特率		1		MHz
工作环境温度	0	25	90	℃
储存温度	-10	25	115	℃
编码器 分辨率（单圈）		16		Bits
编码器 分辨率（多圈）		16		Bits

ESD Rating	±1000			V
开关频率	19.8	20	20.2	KHz

1.2.2 LTH3.2ML-RR60 驱动器（CAN 版本）

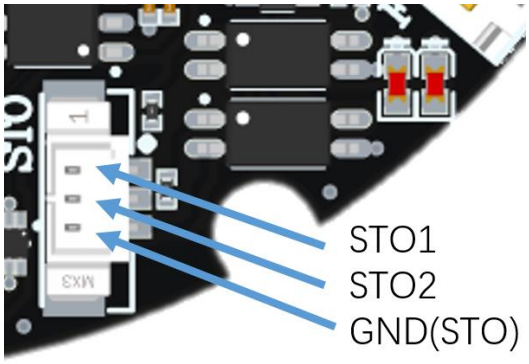


电气参数同上（LTH3.2MW-RR60）

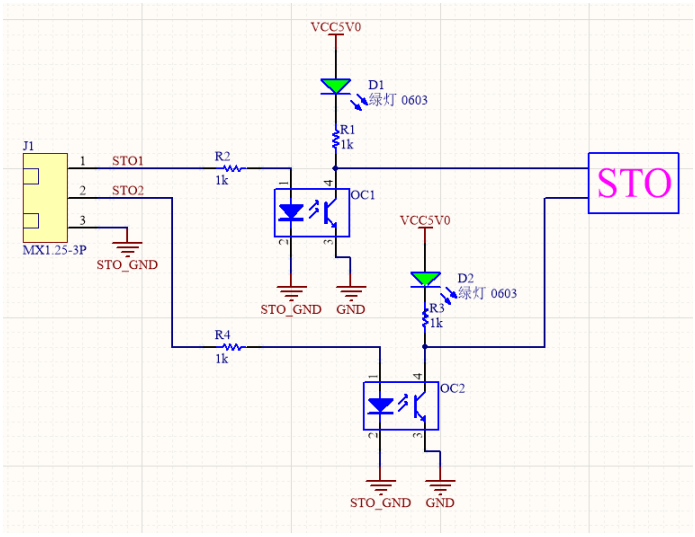
3 Safe Torque Off 机制

若您的产品上没有 STO 插座或没有贴装 STO 插座，请忽视本章。

本产品具备了 Safe Torque Off 功能，以下简称 STO。本产品的 STO 是一个纯硬件的保护措施，可以在出现危险的时刻及时切断驱动板的动力输出。在本产品上，STO 的硬件接口为一个 MX1.25-3P 插座。



本产品的 STO 具有光耦隔离，最大隔离电压 2000v，此外 STO 的 GND 和驱动板上的 GND 是互相隔离的，因此您可以放心地接入保护措施，如急停开关。



LTG 驱动板 STO 实现原理简图

以下表格详细地说明了 STO 的工作机制，请仔细阅读：

STO1	STO2	LED1	LED2	电机状态
上电（12-24v）	上电（12-24v）	亮	亮	可以输出扭矩
下电（0v）	上电（12-24v）	不亮	亮	不输出扭矩
上电（12-24v）	下电（0v）	亮	不亮	不输出扭矩
下电（0v）	下电（0v）	不亮	不亮	不输出扭矩

*此外，将 GND 与 STO GND 短接在一起是允许的。

4 开机自检

每当驱动板上电时，驱动板会发出“滴滴”两声，示意上电。随后驱动板会执行一次硬件健康开机自检，如果检测到驱动板健康状况良好，则会发出“嘀嘀嘀”三声，此后驱动板可以正常运行。

如检测到硬件损伤，则驱动板不会驱动电机运行，并发出四声长短不一的提示音（具体解释请见下图），会接收/返回 CAN 报文，并反馈不可清除故障码（故障码大于 128 时将无

法通过 Rest State 命令清除)，同时状态指示灯(蓝灯)按一定规律闪烁，蜂鸣器也将随蓝灯一起发声指示故障。此时移除电源，请按照以下流程检查:

状态指示灯/蜂鸣器闪烁情况	故障类型	处理方法
长-短-短-短	CAN 收发器硬件故障	请联系客服，返厂维修
长-长-短-短	检测到非法固件	请联系客服，返厂维修
短-长-短-短	角度传感器硬件故障	确保驱动板牢固地固定在电机上，三颗安装螺丝以及垫片都完好且旋紧，电机磁环无破损，若无法解决，请联系客服
短-短-长-短	驱动器硬件故障	请检查驱动功率管表面是否完好，驱动板表面是否干净无异物，移除主供电，再接入，若无法解决，请联系客服
短-短-短-长	功率管硬件故障	请检查驱动功率管表面是否完好，驱动板表面是否干净无异物，以及三相动力线是否与驱动板焊接牢固，若无法解决，请联系客服
长-长-长-短	EtherCAT 通信故障	RR-HA-40 及 RR-HA-60 用户请检查 EtherCAT 通信小板与底层主板连接是否牢固，若无法解决，请联系客服

5 校准

当您重新安装驱动板后（**强烈不建议您自行拆装驱动板，编码器校准和电角度校准需要专业的操作，您自行操作可能会永久性损坏驱动板，本公司对此类情况概不负责。**），您必须再次校准驱动才能继续使用。如果您使用的驱动板是对心的编码器，可以跳过 5.1 编码器校准直接自行执行 5.2 电角度校准。若您使用的是中空的编码器，则需要先进行编码器校准，再执行 5.2 电角度校准，不建议您独立完成，建议联系客服在线完成。

校准也可以参考视频：【Motorevo 关节驱动器保姆教程】 【精准空降到 12:16】
https://www.bilibili.com/video/BV1Uh4y1C7Xw/?share_source=copy_web&vd_source=0be83eaef43126aba089ca5f7d82e5de&t=736

5.1 编码器校准

请联系客服在线支持。

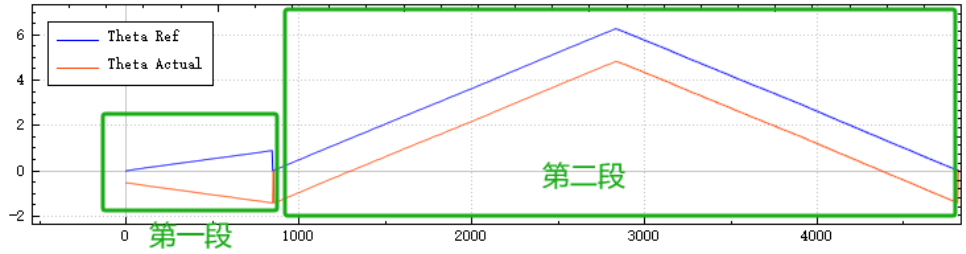
5.2 电角度校准

校准流程如下：
请确保电机处于 Rest Mode 下，电机输出端无任何外加负载，插入 USB 数据线，确保动力电主供电稳定且电压高于 24V（最好是 24V），进入 Motor Parameter Studio 界面，点击 UNCONNECT 按键，若显示绿色 CONNECT 字样，则说明 USB 连接成功。
在 USB 设备连接成功后，在 Encoder Calibration 窗口，点击 Start 按钮，等待两个进度条均如下图所示显示 100%时，即校准完毕，校准时电流约为 0.2~0.4A，校准结束后，电流恢复至 0.03A 左右，移除 USB 数据线，断开 24V 供电再接入，即完成一次校准。



曲线说明：

Electrical Angle Offset Calibration



如图所示，第一段的方向可以相同也可以相反，是用于校准电机的方;，第二段曲线中蓝线和红线应该严格平行，同时红线应该平直。

异常处理:

若第二段中红线和蓝线不平行，可以检查电机参数中的极对数 NPP 是否正确。

6 控制与通信

6.1 状态定义

动力单元共有两种运行状态:

- **Rest State:**

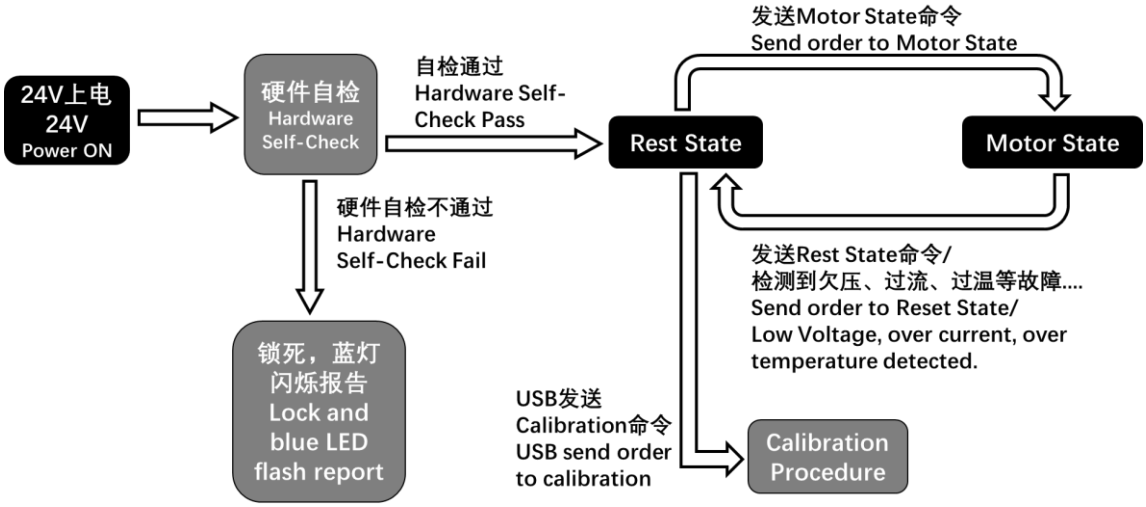
休眠模式，在 Reset 状态下，工作电流约为 0.02A（24V），状态指示灯不常亮，功率管不开关，驱动板 ABC 三相均输出 24V，此时电机处于低功耗待机状态，对所有收到的 CAN 报文、EtherCAT 报文进行回应，角度编码器正常工作。

- **Motor State:**

运行模式，在 Motor 状态下，状态指示灯（蓝灯）常亮，功率管将会开始工作，此时电机进入正常运动工作状态，此时电机的工况由 CAN 发送命令直接控制，在此状态下，板子上 ABC 三相输出矢量调制 PWM 波，对所有收到的 CAN 报文、EtherCAT 报文进行回应，角度编码器正常工作。

在电机运行过程中，如果检测到欠他过流等故障，电机将自动退出 Motor State，进入 Rest State，此时您需要发送 Enter Rest State 命令，清除故障码，才能使电机再进入 Motor State，若您没有清除故障码，即使发送了 Enter Motor State 命令，驱动也不会相应并进入 Motor State。

状态机切换如下图:



通过以下命令，可以让驱动器在 Motor State 和 Rest State 之前切换。

1. 命令执行器进入 **Motor State**，请发送 **Enter Motor State** 命令:
   Enter Motor State 帧为 CAN 标准帧，帧 ID 为 Motor ID（在 Torque4 Mode 下为 100），波特率为 1Mhz ：

Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7
0xFF	0xFF	0xFF	0xFF	0xFF	0xFF	0xFF	0xFC

2. 命令执行器进入 **Rest State**，请发送 **Enter Rest State** 命令:
   Enter Rest State 帧为 CAN 标准帧，帧 ID 为 Motor ID（在 Torque4 Mode 下为 100），波特率为 1Mhz ：

Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7
0xFF	0xFF	0xFF	0xFF	0xFF	0xFF	0xFF	0xFD

3. 命令执行器设置当前角度为零位，请发送 **Set Zero Position** 命令:
   Set Zero Position 帧为 CAN 标准帧，帧 ID 为 Motor ID（在 Torque4 Mode 下为 100），波特率为 1Mhz ：

Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7
0xFF	0xFF	0xFF	0xFF	0xFF	0xFF	0xFF	0xFE

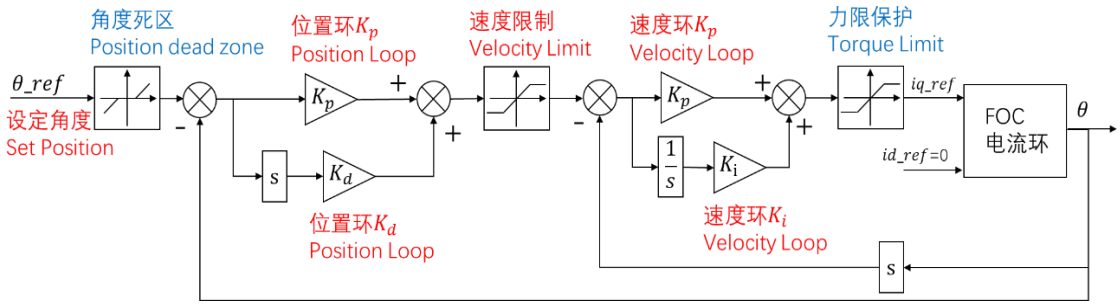
6.2 控制模式简介

Motorevo 系列驱动集成了多种控制模式，您可以选择您需要的模式进行使用，您可以在 6.7.3 章中查看如何修改控制模式。**注意，在使用时，请一定要确认驱动 Flash 中设定的控制模式和 CAN 数据帧的控制模式一致。若不一致，可能会导致电机异常运动，甚至失控，造成生命财产损失！**

6.2.1 伺服模式 Servo Mode

本驱动板的伺服控制模式基于电流环、速度环、位置环三闭环控制器，在此基础上使用非线性滑膜观测器(ULSMO)对扰动进行观测和建模，并施加前馈转矩以抵消扰动，并采用了在线参数辨识技术，实时检测电机运行状态，动态调整控制器。

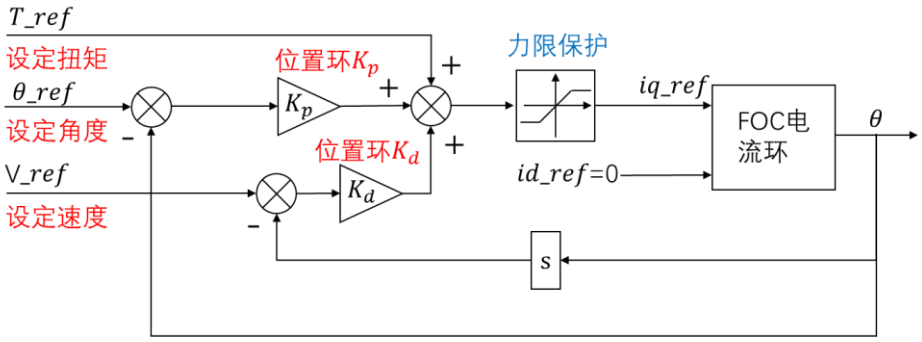
总而言之，该模式下具有过渡速度可控、抗扰性强、极小静差以及高带宽等特色，伺服控制模式的简略控制框图如下:



*其中红色的参数均需要通过 CAN 通信的方式设定，蓝色的参数需要通过 USB 连接 PC，在 Motorevo Studio™中修改,参见第 6.7 章。在此模式下，若使用 CAN 协议对执行器进行控制，请参考 6.3.1 节使用 CAN 控制电机在伺服模式运行。

6.2.2 力位混合模式 Torque-Position-Mix Control Mode

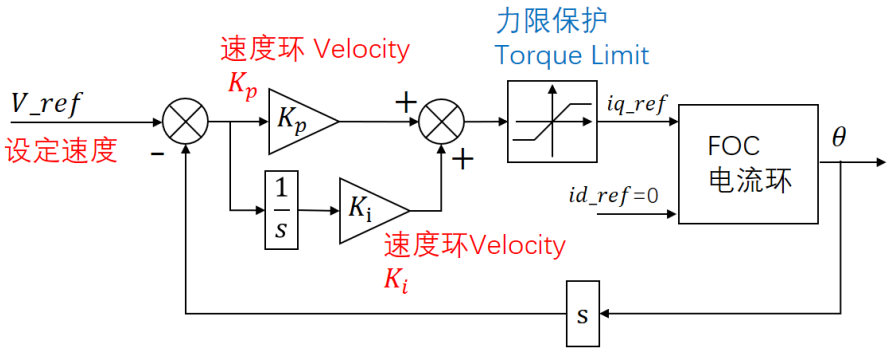
本驱动板的力位混合控制模式基于电流环、位置环双闭环控制器，力位混合控制模式的简略控制框图如下:



*其中红色的参数均需要通过 CAN 通信的方式设定，蓝色的参数需要通过 USB 连接 PC，在 Motorevo Studio™中修改,参见第 6.7 章。在此模式下，若使用 CAN 协议对执行器进行控制，请参考 6.3.2 节使用 CAN 控制电机在力位混合模式运行。

6.2.3 速度模式 Velocity Mode

本驱动板的速度控制模式基于电流环、速度环双闭环控制器，在此基础上也使用非线性滑膜观测器(ULSMO)对扰动进行观测和建模，在突然施加负载的情况下较传统的控制器降速更低，能提供一个持续地、稳定地转速。



*其中红色的参数均需要通过 CAN 通信的方式设定，蓝色的参数需要通过 USB 连接 PC，在 Motorevo Studio™中修改,参见第 6.7 章。在此模式下，若使用 CAN 协议对执行器进行控制，请参考 6.3.3 使用 CAN 控制电机在速度模式（Velocity Mode）运行。

6.2.4 力矩模式 Torque Mode

在力矩模式下，可以用一帧 CAN 指令对一个电机进行控制，直接设置期望力矩。在此模式下，若使用 CAN 协议对执行器进行控制，请参考 6.3.4 使用 CAN 控制电机在力矩模式（Torque Mode）运行

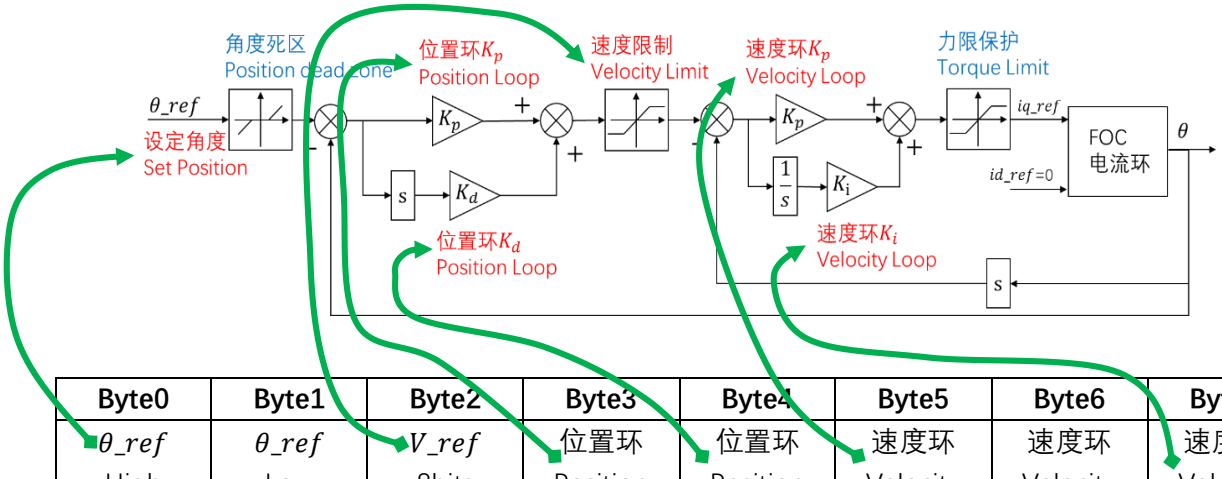
6.2.5 力矩 4 模式 Torque4 Mode

在力矩 4 模式下，可以用一帧 CAN 指令对 4 个电机同时控制力矩。相比于 Torque Mode，Torque4 Mode 的通信效率提升了 4 倍。在此模式下，若使用 CAN 协议对执行器进行控制，请参考 6.3.5 使用 CAN 控制电机在力矩 4 模式（Torque4 Mode）运行。

6.3 CAN 帧格式定义

6.3.1 使用 CAN 控制电机在伺服模式（Servo Mode）运行

控制帧为 CAN 标准帧，帧 ID 为 Motor ID，波特率为 1Mhz，控制帧和控制框图的映射关系定义如下 ：



Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7
θ_ref High 8bits	θ_ref Low 8bits	V_ref 8bits	位置环 Position K_p 8bits	位置环 Position K_d 8bits	速度环 Velocity K_p 8bits	速度环 Velocity K_d 8bits	速度环 Velocity K_i 8bits

其中，每个参数的映射范围如下:

参数 Parameter	比特数 Bits	映射范围 Mapping range	单位 Units
θ_ref Set Position	16bits	[0x0000~0xFFFF] map to: **[CAN COM Theta MIN ~ CAN COM Theta MAX]** Default: -12.5 ~ +12.5	rad
V_ref Transition Velocity	8bits	[0x00~0xFF] map to: **[CAN COM Velocity MIN ~ CAN COM Velocity MAX]** Default: -10.0 ~ +10	rad/s
位置环 Position Loop K_p	8bits	[0x00~0xFF] map to: **[CAN COM Kp MIN ~ CAN COM Kp MAX]** Default: 0.0 ~ 250.0	/
位置环 Position Loop K_d	8bits	[0x00~0xFF] map to: **[CAN COM Kd MIN ~ CAN COM Kd MAX]** Default: 0.0 ~ 50.0	/
速度环 Velocity Loop K_p	8bits	[0x00~0xFF] map to: **CAN COM Kp MIN ~ CAN COM Kp MAX** Default: 0.0 ~ 250.0	/

速度环 Velocity Loop K_{d}	8bits	[0x00~0xFF] map to: **[CAN COM Kd MIN ~ CAN COM Kd MAX]** Default: 0.0 ~ 50.0	/
速度环 Velocity Loop K_{i}	8bits	[0x00~0xFF] map to: **[CAN COM Ki MIN ~ CAN COM Ki MAX]** Default: 0.0 ~ 0.05	/

实例 Example: $\theta_{ref} = 3.14(rad)$, $V_{ref} = 2.0(rad/s)$,

位置环 Position Loop: $K_p = 15.0$

位置环 Position Loop: $K_d = 4.5$

速度环 Velocity Loop: $K_p = 50.0$

速度环 Velocity Loop: $K_{\mathrm{d}} = 0$

速度环 Velocity Loop: $K_{\mathrm{i}} = 0.001$

(使用默认参数范围)

转化为控制帧为:

Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7
0xA0	0x26	0x99	0x0F	0x16	0x33	0x00	0x05

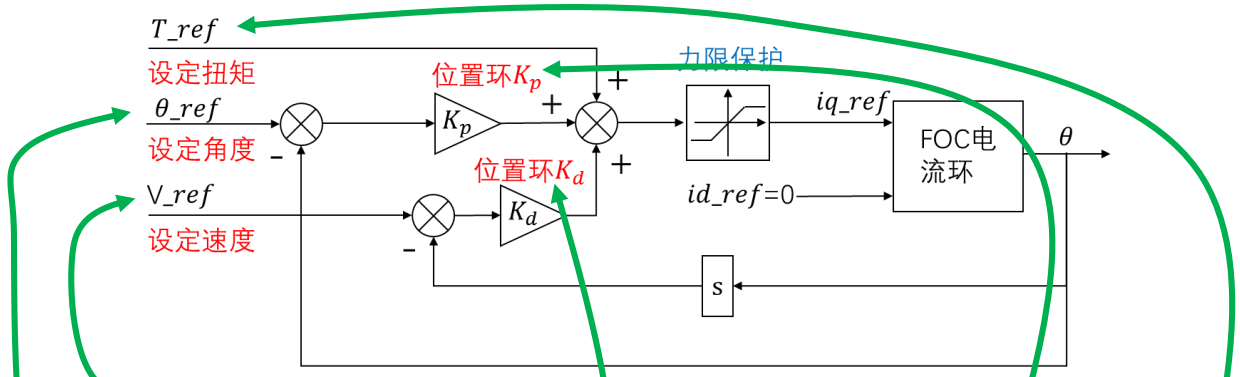
伺服模式下上位机 CAN 指令封包例程

```
/* @brief:发送函数 Package Function
   @param:
    f_posi: 电机的目标角度，单位为弧度
    The target angle of the motor, in radians
    f_velo: 电机到达目标角度的最大转速，单位为 rad/s
    The maximum speed of the motor reaching the target angle, the
    unit is rad/s
    f_kp:   电机位置环 Kp Position Loop Kp
    f_kd:   电机位置环 kd Position Loop Kd
    f_kp_i: 电机速度环 Kp Velocity Loop Kp
    f_ki_i: 电机速度环 Ki Velocity Loop Ki
    f_kd_i: 电机速度环 Kd Velocity Loop Kd
    id:     电机 id  CAN ID */
void Protocsend(float f_posi,float f_velo,float f_kp,float f_kd,
float f_kp_i,float f_kd_i,float f_ki_i,uint8_t id)
{
    int p = float_to_uint(f_posi, P_MIN, P_MAX, 16);
    int v = float_to_uint(f_velo, V_MIN, V_MAX, 8);
    int kp = float_to_uint(f_kp, KP_MIN, KP_MAX, 8);
    int kd = float_to_uint(f_kd, KD_MIN, KD_MAX, 8);
    int kp_i = float_to_uint(f_kp_i,KP_MIN,KP_MAX,8);
    int kd_i = float_to_uint(f_kd_i,KD_MIN,KD_MAX,8);
    int ki_i = float_to_uint(f_ki_i,KI_MIN,KI_MAX,8);

    uint8_t data[8];
    data[0] = p >> 8;
    data[1] = p & 0xFF;
    data[2] = v;
    data[3] = kp;
    data[4] = kd;
    data[5] = kp_i;
    data[6] = kd_i;
    data[7] = ki_i;
    transmitFromCanInterface(data,id);
}
```

6.3.2 使用 CAN 控制电机在力位混合模式（P-T-M Mode）运行

控制帧为 CAN 标准帧，帧 ID 为 Motor ID，波特率为 1Mhz，控制帧和控制框图的映射关系定义如下 ：



Byte0	Byte1	Byte2	Byte3	
θ_ref High 8bits	θ_ref Low 8bits	V_ref High 8bits	V_ref Low 4 bits (High 4 bits of Byte3)	K_p High 4 bits (Low 4 bits of Byte3)

Byte4	Byte5	Byte6		Byte7
K_p Low 8bits	K_d High 8bits	K_d Low 4 bits (High 4 bits of Byte6)	T_ref High 4 bits (Low 4 bits of Byte6)	T_ref Low 8 bits

其中，每个参数的映射范围如下:

参数 Parameter	比特数 Bits	映射范围 Mapping range	单位 Units
θ_ref Set Position	16bits	[0x0000~0xFFFF] map to: **[CAN COM Theta MIN ~ CAN COM Theta MAX]** Default: -12.5 ~ +12.5	rad
V_ref Transition Velocity	12bits	[0x000~0xFFF] map to: **[CAN COM Velocity MIN ~ CAN COM Velocity MAX]** Default: -10.0 ~ +10	rad/s
K_p	12bits	[0x000~0xFFF] map to: **[CAN COM Kp MIN ~ CAN COM Kp MAX]** Default: 0.0 ~ 250.0	/
K_d	12bits	[0x000~0xFFF] map to: **[CAN COM Kd MIN ~ CAN COM Kd MAX]** Default: 0.0 ~ 50.0	/

T_ref	12bits	[0x000~0xFFF] map to: **[CAN COM Torque MIN ~ CAN COM Torque MAX]** Default：-50.0 ~ 50.0	Nm

***实例 1* 位置 PD 控制**

$\theta_{ref} = 3.14(rad)$

$V_{ref} = 0.0(rad/s)$,

$K_p = 15.0$

$K_d = 4.5$

$T_ref = 0$

(使用默认参数范围)

转化为控制帧为:

Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7
0xA0	0x26	0x7F	0xF0	0xF5	0x17	0x07	0xFF

实例 2 力位混合控制

$\theta_{ref} = 3.14(rad)$

$V_{ref} = 0.0\left(\frac{rad}{s}\right)$,

$K_p = 15.0$

$K_d = 4.5$

$T_ref = 5$

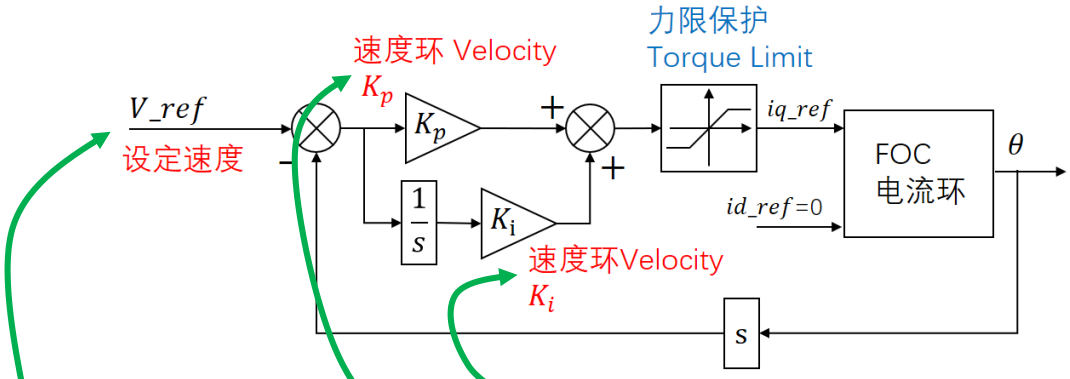
(使用默认参数范围)

转化为控制帧为:

Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7
0xA0	0x26	0x7F	0xF0	0xF5	0x17	0x08	0xCC

6.3.3 使用 CAN 控制电机在速度模式（Velocity Mode）运行

控制帧为 CAN 标准帧，帧 ID 为 Motor ID，波特率为 1Mhz，控制帧和控制框图的映射关系定义如下 ：



Byte0	Byte1	Byte2	Byte3	
V_ref High 8bits	V_ref Low 8bits	K_p High 8bits	K_p Low 4 bits (High 4 bits of Byte3)	K_d High 4 bits (Low 4 bits of Byte3)

Byte4	Byte5	Byte6		Byte7
K_d Low 8bits	K_i High 8bits	K_i Low 4 bits (High 4 bits of Byte6)	无意义	0xAC Identification

其中，每个参数的映射范围如下:

参数 Parameter	比特数 Bits	映射范围 Mapping range	单位 Units
V_ref Set Velocity	16bits	[0x0000~0xFFFF] map to: **[CAN COM Velocity MIN ~ CAN COM Velocity MAX]** Default: -10.0 ~ +10.0	rad/s
K_p	12bits	[0x000~0xFFF] map to: **[CAN COM Kp MIN ~ CAN COM Kp MAX]** Default: 0.0 ~ 250.0	/
K_d	12bits	[0x000~0xFFF] map to: **[CAN COM Kd MIN ~ CAN COM Kd MAX]** Default: 0.0 ~ 50.0	/
K_i	12bits	[0x000~0xFFF] map to: **[CAN COM Ki MIN ~ CAN COM Ki MAX]** Default: 0.0 ~ 0.05	/

6.3.4 使用 CAN 控制电机在力矩模式（Torque Mode）运行

控制帧为 CAN 标准帧，帧 ID 为 Motor ID，波特率为 1Mhz，控制帧和控制框图的映射关系定义如下 ：

Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7
T_ref High 8 bits	T_ref Low 8bits	无意义	无意义	无意义	无意义	无意义	0xAB Identification

其中，每个参数的映射范围如下:

参数 Parameter	比特数 Bits	映射范围 Mapping range	单位 Units
T_ref Set Torque	16bits	[0x0000~0xFFFF] map to: **[CAN COM Torque MIN ~ CAN COM Torque MAX]** Default： -50.0 ~ 50.0	Nm

6.3.5 使用 CAN 控制电机在力矩 4 模式（Torque4 Mode）运行

控制帧为 CAN 标准帧，帧 ID 为 100，波特率为 1Mhz，控制帧和控制框图的映射关系定义如下 ：

Byte0	Byte1	Byte2	Byte3
T_ref High 8 bits 对于(Motor ID-1%4)=0 的电机，如 1,5,9,…	T_ref Low8 bits 对于(Motor ID-1%4)=0 的电机，如 1,5,9,…	T_ref High 8 bits 对于(Motor ID-1%4)=1 的电机，如 2,6,10,…	T_ref Low8 bits 对于(Motor ID-1%4)=1 的电机，如 2,6,10,…
Byte4	**Byte5**	**Byte6**	**Byte7**
T_ref High 8 bits 对于(Motor ID-1%4)=2 的电机，如 3,7,11,…	T_ref Low8 bits 对于(Motor ID-1%4)=2 的电机，如 3,7,11,…	T_ref High 8 bits 对于(Motor ID-1%4)=3 的电机，如 4,8,12,…	T_ref Low8 bits 对于(Motor ID-1%4)=3 的电机，如 4,8,12,…

其中，每个参数的映射范围如下:

参数 Parameter	比特数 Bits	映射范围 Mapping range	单位 Units
T_ref Set Torque	16bits	[0x0000~0xFFFF] map to: **[CAN COM Torque MIN ~ CAN COM Torque MAX]** Default： -50.0 ~ 50.0	Nm

6.5 CAN 通信反馈

6.5.1 反馈帧格式

无论在什么状态和模式下，驱动板在收到报文标识符为 **CAN COM ID** 的报文时都会进行一次应答，反馈当前电机的位置(rad)，速度(rad/s)，扭矩(N·m)。应答的报文标识符恒定本机 CAN ID，波特率为 1Mbps，DLC 为 8 字节。您可以在 6.7 章查看如何修改 **CAN COM ID。**

Byte0	Byte1	Byte2	Byte3	Byte4		Byte5	Byte6	Byte7
状态字 Status	位置高 8bits	位置低 8bits	速度高 8bits	速度低 4bits（Byte4 高 4 字节）	输出力矩高 4bits（Byte4 低 4 字节）	输出力矩 低 8bits	故障码	驱动板温度

6.5.2 反馈帧状态解包

状态字（Status）

定义如下：

值	含义
0x00	当前处于 Reset Status
0x01	当前处于 Motor Status，且控制模式为伺服模式
0x02	当前处于 Motor Status，且控制模式为力位混合模式
0x03	当前处于 Motor Status，且控制模式为速度模式
0x04	当前处于 Motor Status，且控制模式为力矩模式
0x05	当前处于 Motor Status，且控制模式为 Torque 4 混合模式

位置（Position）共 16 bits:

0x0000~0xFFFF 对应 **CAN COM Theta MIN ~ CAN COM Theta MAX**
默认值 Default： -12.5 rad ~+12.5 rad

速度（Velocity）共 12 bits:

0x000~0xFFF 对应 **CAN COM Velocity MIN ~ CAN COM Velocity MAX**
默认值 Default： -10 rad/s ~+10 rad/s

力矩（Torque）共 12 bits:

0x000~0xFFF 对应 **CAN COM Torque MIN ~ CAN COM Torque MAX**
默认值 Default:　-50Nm~+50Nm

驱动板温度 共 8 bits:

数据类型为 unsigned char，单位为摄氏度

故障码 共 8 bits:

一旦驱动板检测到故障，将会从 Motor State 自动切回 Rest State 以保护驱动器和电机，同时将返回故障码，提示用户电机运行异常。此时您应仔细检查电机运行工况是否正常，在排除异常情况后，发送 Enter Rest State 命令清除故障码，再发送 Enter Motor State 命令让电机重新恢复运行（**注意，大于 128 的故障码为开机自检检测出的故障码，无法用该方法清除**）。
故障码定义如下:

故障码 DEC	故障码 HEX	故障名称	可能原因	解决方案
0	0x00	无故障		
1	0x01	直流母线电压过压	1、 电机制动或倒拉反转导致母线电压上升。 2、 电源毛刺。	1、 在母线上增加电解电容，降低母线电压波动。 2、 通过上位机修改电压保护阈值（请慎重修改，盲目扩大保护阈值可能会增加电机损坏风险）。
2	0x02	直流母线电压欠压	1、 电流过大导致电源电压降低。 2、 电源毛刺。	1、 在母线上增加电解电容，降低母线电压波动。 2、 检查驱动板是否有短路情况。 3、 通过上位机修改电压保护阈值（请慎重修改，盲目扩大保护阈值可能会增加电机损坏风险）。
3	0x03	编码器电角度故障	驱动板安装不稳，或拆装驱动板后没	重启后如果还报该故障，请重新校准驱动板

			有校准编码器	
4	0x04	DRV 驱动器故障	硬件过载等	重启后如果还报该故障，可能驱动板已经发生硬件损坏，请联系客服售后维修。
5	0x05	直流母线电流过流	负载过大	1、 减小负载。 2、 通过上位机修改电流保护阈值（请慎重修改，盲目扩大保护阈值可能会增加电机损坏风险）。
6	0x06	电机 A 相电流过流	负载过大	1、 减小负载。 2、 2、通过上位机修改电流保护阈值（请慎重修改，盲目扩大保护阈值可能会增加电机损坏风险）。
7	0x07	电机 B 相电流过流	负载过大	同故障码 6
8	0x08	电机 C 相电流过流	负载过大	同故障码 6
9	0x09	驱动板温度过温	负载过大或环境温度过高。	1、 减小负载。 2、 通过上位机修改电流保护阈值（请慎重修改，盲目扩大保护阈值可能会增加电机损坏风险）。 3、 增加散热条件，如风冷、液冷等截至散热。
10	0x0A	电机线圈过温	负载过大或环境温度过高。	1、 减小负载。 2、 通过上位机修改电流保护阈值（请慎重修改，盲目扩大保护阈值可能会增加电机损坏风险）。 增加散热条件，如风冷、液冷等截至散热。
11	0x0B	编码器故障	多重原因	1、 重启，若重启后还报该错误则按照如下检查。 2、 检查编码器、码盘或磁环表面是否有异物， 3、 重新校准电机。
12	0x0C	电流传感器故障	硬件损坏	1、 检查驱动板正面、背

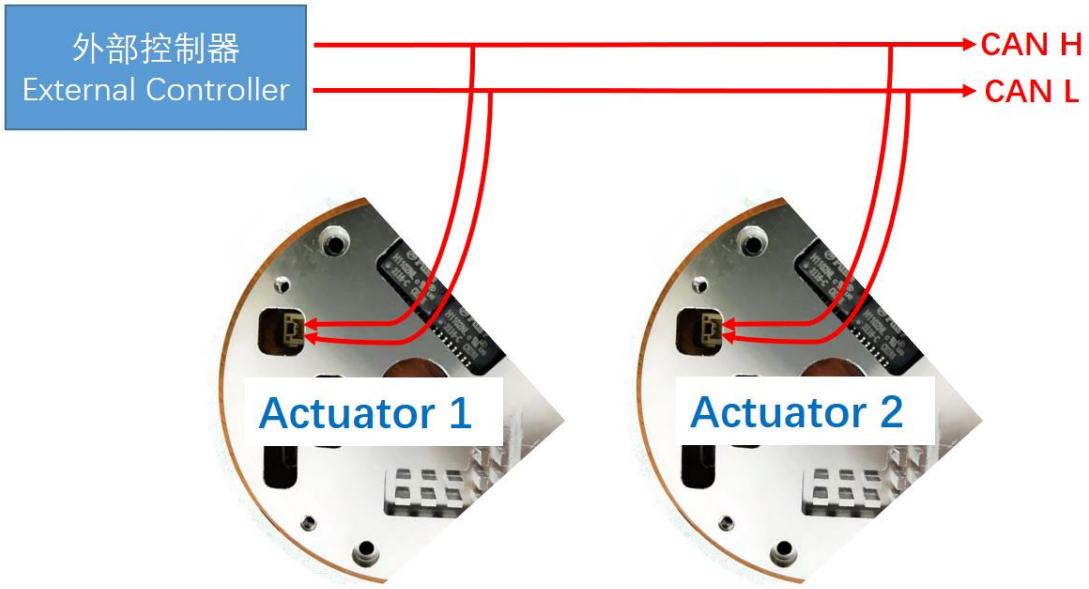
				面有无异物。 2、 检查采样电阻外观是否正常、是否破损。 3、 硬件损坏，此情况下需联系售后维修。
13	0x0D	输出轴实际角度超过通信范围：**CAN COM Theta MIN ~ CAN COM Theta MAX**	运行故障，或编码器异常	如果是运行过程中意外超限，请手动将电机输出轴位置调回通信范围内。或检查编码器，可能是编码器读值异常
14	0x0E	输出轴速度超过通信范围 **CAN COM Velocity MIN ~ CAN COM Velocity MAX**	运行故障，或编码器异常	确保通信值范围合理
15	0x0F	堵转保护	电机电枢电流(Iq)大于 **Stuck Current**，同时电机速度小于 **Stuck Velocity**，持续时间超过 **Stuck Time** 后触发	检查电机运行工况，排除堵转工况。或可能是电角度 offset 丢失，导致电机电角度零位错误。
16	0x10	CAN 通讯丢失	超过 **CAN COM TIMEOUT** 时间没有收到 CAN 数据帧时出发	检查 CAN 通讯线路。
以下为不可清零故障码（无法用 Reset Status 命令清除），由开机自检设定				
129	0x81	离轴/对心多圈绝对值编码器接口帧头校验失败	编码器芯片损坏	重启，若还是失败则请联系售后工程师
130	0x82	对心多圈绝对值编码器多圈故障	对心多圈绝对值编码器系统异常	在 USB 界面重新初始化多圈编码器后，重启，若还失败则请联系售后工程师
131	0x83	对心多圈绝对值编码器多圈接口故障		
132	0x84	对心多圈绝对值编码器外部输入故障		
133	0x85	对心多圈绝对值编码器读值故障		
134	0x86	对心多圈绝对值编码器 ERR_OFFS		
135	0x87	对心多圈绝对值编		

		码器 ERR_CFG		
136	0x88	检测到非法固件		请联系售后工程师
137	0x89	集成式栅极驱动器初始化失败	集成式栅极驱动器损坏	请联系售后工程师

6.6 多执行器级联运行

6.6.1 CAN 实现多执行器级联运行

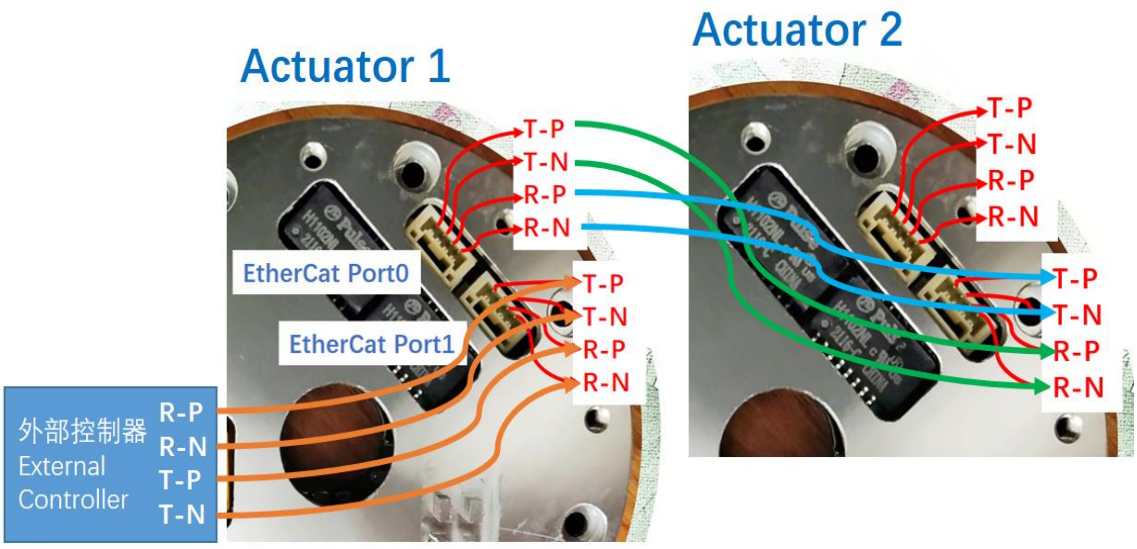
对于只有一个 CAN 座子的驱动器，可以按照如下方式链接：

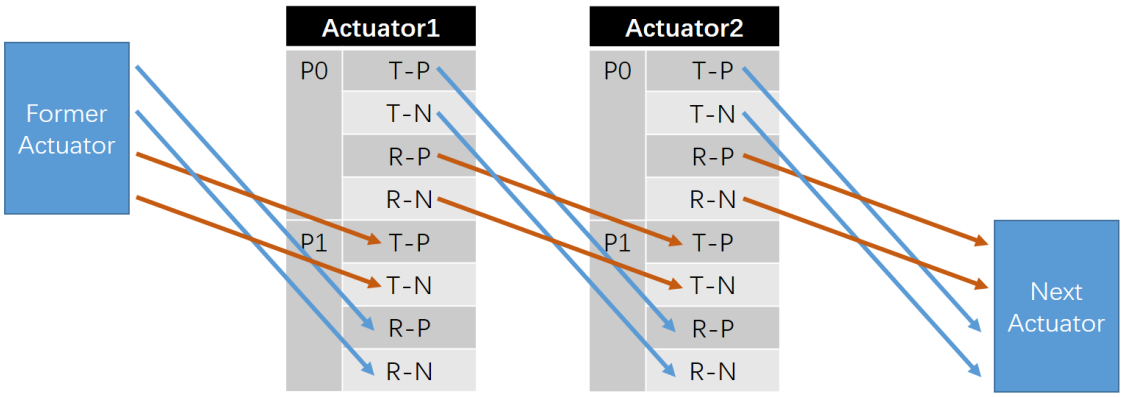


对于有两个 CAN 座子的驱动器，可以采用一进一出的方式进行连接，大大降低了用户的接线难度。

6.6.2 EtherCAT 实现多执行器级联运行

若使用 EtherCAT 控制多台执行器，请严格按照以下连接方式连接。
请注意，TP 与 TN，RP 与 RN 间尽量使用双绞线连接，如无法实现，请确保 P 信号线与 N 信号线的长度尽量一致!





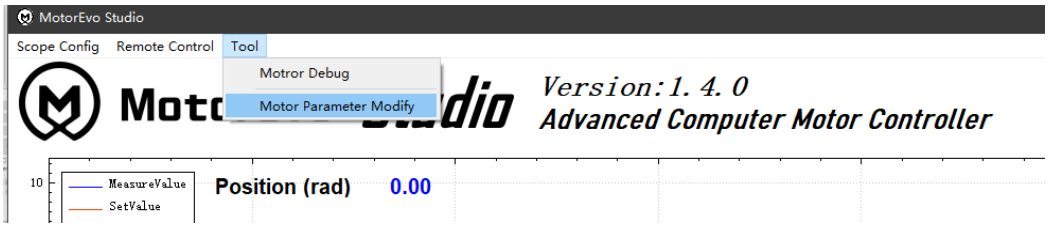
6.7 使用 MParameter Studio 修改电机参数

本章可以参考视频：【Motorevo 关节驱动器保姆教程】 【精准空降到 01:25】 https://www.bilibili.com/video/BV1Uh4y1C7Xw/?share_source=copy_web&vd_source=0be83eaef43126aba089ca5f7d82e5de&t=85，帮助您尽快上手参数修改。

6.7.1 建立连接与保存

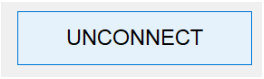
Step1.

按下图所示，从 Motorevo Studio™软件中进入 MParameter Studio 界面，在确保没有 24V 供电的情况下，插入 USB 线。



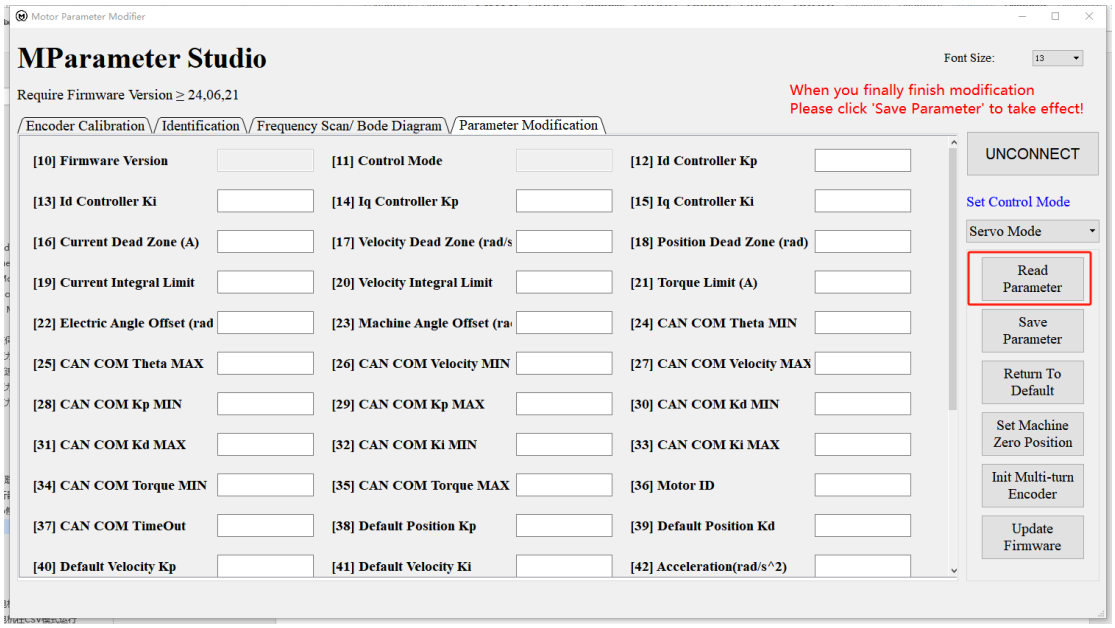
Step2.

插入 USB 线后点击 UNCONNECT 等待软件自动识别并连接，变成绿色 CONNECT 字样则说明驱动板连接成功。



Step3.

点击 Read Parameter 按键，从驱动板中读取当前参数。

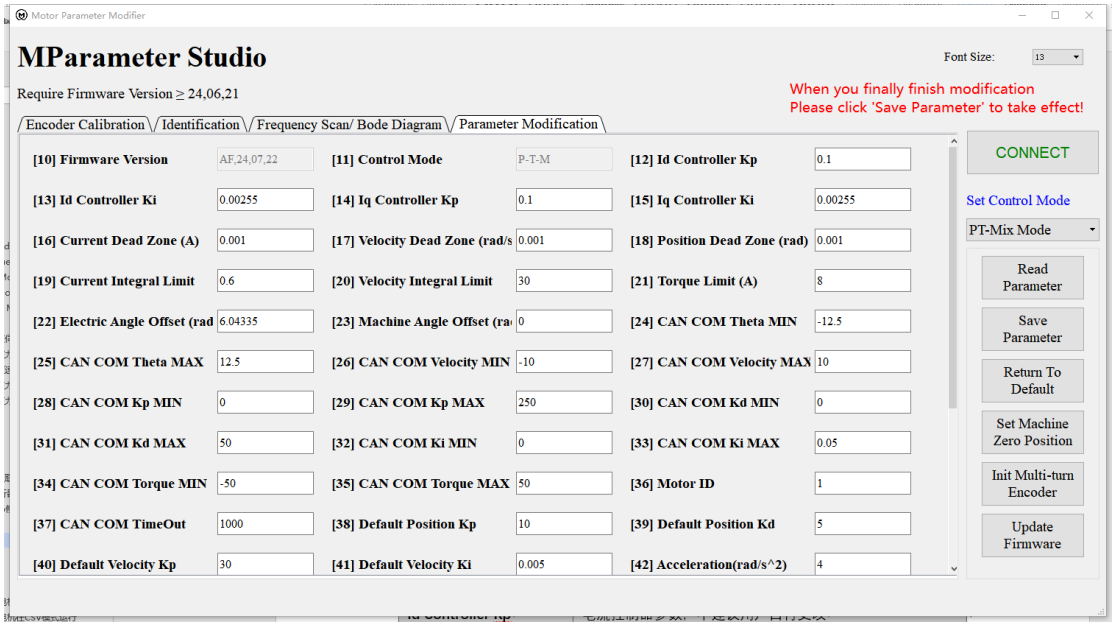


Step4.

您便可以修改参数，**每次修改完后请务必按下键盘上的回车按键**，全部修改完后，请点击 Save Flash 命令，电机会将您设置的这组参数永久地储存在内部。

提示：点击 Save Flash 后请等待 1~2s，让驱动板完成 Flash 擦写，期间请尽量保持连接和驱动板的稳定，保存完成后您可以再按 Read Parameter 读取一次，以检查驱动板是否正确保存了您的参数。

6.7.2 参数说明



该界面下的参数解释如下：

参数名	Index (Dec)	数据类型	单位	解释
Control Mode	11	Int32	/	电机控制模式，定义如下： ● 0x01：伺服模式(Servo) ● 0x02：力位混合模式(Torque-Position Mixed) ● 0x03：速度模式(Velocity) ● 0x04：力矩模式(Torque)
Id Controller Kp	12	Float32	V/A	FOC 电流环 D 轴 PI 控制器比例项 Kp
Id Controller Ki	13	Float32	Hz	FOC 电流环 D 轴 PI 控制器积分项 Ki
Iq Controller Kp	14	Float32	V/A	FOC 电流环 Q 轴 PI 控制器比例项 Kp
Iq Controller Ki	15	Float32	Hz	FOC 电流环 Q 轴 PI 控制器积分项 Ki
Current DeadZone	16	Float32	A	电流环死区，当电流环 DQ 轴电流 error 小于此值时，电流环不输出
Velocity DeadZone	17	Float32	Rad/s	速度环死区，当设定速度和实际速度之差小于此值时，速度环不输出。

Position DeadZone	18	Float32	Rad	位置环死区，当设定位置和实际位置之差小于此值时，位置环不输出。
Current Integral Limit	19	Float32	/	电流环积分限幅系数，将电流环积分的输出限幅在 **Current Integral Limit** ×电源电压的范围内
Velocity Integral Limit	20	Float32	Nm	速度环积分限幅，将速度环输出扭矩的积分部分限制在此范围内
Torque Limit	21	Float32	Nm	控制器力限保护，驱动在任何模式下都不会输出高于此值的扭矩。
Electric Angle Offset	22	Float32	Rad	电机转子实际机械位置与三项坐标系 A 轴角度之差。严禁随意修改，该值错误会导致电机烧毁或失控跑飞！该值仅由电机的 Calibration 命令校准。
Machine Angle Offset	23	Float32	Rad	设定驱动器机械零位，驱动器输出角度为：测量值 - **Machine Angle Offset**，若要设置当前角度为机械零位，仅需将 **Machine Angle Offset** 设定为当前角度值即可，也可以在本界面右侧按 Set Machine Zero Position 按钮即可。
CAN COM Theta MIN	24	Float32	Rad	CAN 通讯数据范围，详情请参照前文 6.3 章。
CAN COM Theta MAX	25	Float32	Rad	
CAN COM Velocity MIN	26	Float32	Rad/s	
CAN COM Velocity MAX	27	Float32	Rad/s	
CAN COM Kp MIN	28	Float32	Nm/rad	
CAN COM Kp MAX	29	Float32	Nm/rad	
CAN COM Kd MIN	30	Float32		
CAN COM Kd MAX	31	Float32		
CAN COM Ki MIN	32	Float32		
CAN COM Ki MAX	33	Float32		
CAN COM Torque MIN	34	Float32		
CAN COM Torque MAX	35	Float32		

Motor ID	36	Int32	无	设置本机 Motor ID 号，范围为 0~1024
CAN COM Time Out	37	Int32	Ms	设置 CAN 中断超时报警时间，详情请参照前文。
Default Position Kp	38	Float32		驱动板在 EtherCAT 模式下伺服模式的位置环 Kp
Default Position Kd	39	Float32		驱动板在 EtherCAT 模式下伺服模式的位置环 Kd
Default Velocity Kp	40	Float32		驱动板在 EtherCAT 模式下伺服模式的速度环 Kp
Default Velocity Ki	41	Float32		驱动板在 EtherCAT 模式下伺服模式的速度环 Ki
Firmware Version	10	Int32		固件版本号
Acceleration	42	Float32	rad/s^2	驱动在伺服模式下的加速度值。
Torque slope	43	Float32		力矩上升率的限制，此值越大，力矩的最大瞬时变化率越大。注意，此值过大可能会导致很高的瞬间冲击电流，损坏驱动器或电机。
NPP	44	Int32	/	电机极对数，要与电机实际极对数一致，错误极对数会导致驱动或电机烧毁。
Gear Ratio	45	Float32	/	减速器减速比，要与实际减速比一致，错误极对数可能会导致位置显示不准确。
Torque Constant	46	Float32	Nm/A	扭矩常数。驱动器中设定扭矩和反馈扭矩与电流的关系为：$\mathrm{Torque} = \mathrm{I_q} \times \mathrm{Torque\ Constant}$。
Rotate Dir	47	Int32	/	无单位。电机运行正方向设置。只能设置为-1 或+1，规定电机正转的方向。
BUS_OV_LOCK	49	Float32	V	母线电压过压保护阈值，当驱动检测到母线电压高于此值时会从 Motor State 切入 Rest State，并反馈故障码，详情见 6.5.2 故障码。
BUS_UV_LOCK	50	Float32	V	母线电压欠压保护阈值，当驱动检测到母线电压低于此值时会从 Motor State 切入 Rest State，并反馈故障码，详情见 6.5.2 故障码。
PHASE_OC_LOCK	51	Float32	A	相电流过流保护阈值，当驱动检测到任意一相电流绝对值高于此值时会从 Motor State 切入 Rest State，并反

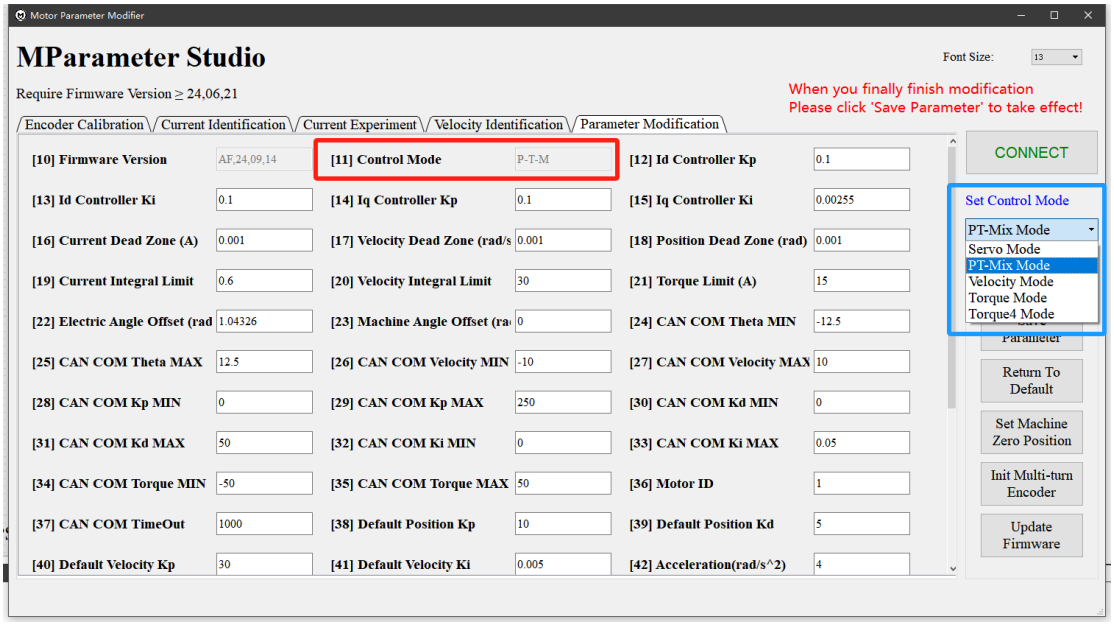
				馈故障码，详情见 6.5.2 故障码。
PCB_OT_LOCK	52	Float32	℃	驱动板温度保护阈值，当驱动检测到温度高于此值时会从 Motor State 切入 Rest State，并反馈故障码，详情见 6.5.2 故障码。
Stuck Current	53	Float32	A	当电机电枢电流(Iq)大于此值，同时速度小于 Stuck Velocity，持续时间超过 Stuck Time 时触发 Stuck 堵转保护，详情见 6.5.2 故障码。
Stuck Velocity	54	Float32	Rad/s	详情同上
Stuck Time	55	Float32	s	详情同上

Protect Switchs	56	Int32	/	保护开关，每个 bit 表示对应保护，1 为打开，0 为关闭：
				// Bit0: 位置超限保护
				// Bit1: 超速保护
				// Bit2: 相电流过流保护
				// Bit3: 过温保护
				// Bit4: 过压欠压保护
				// Bit5: 堵转保护
				// Bit6: 输入轴编码器自检
				// Bit7: CAN 通讯中断保护
				例如，如果全开保护，可以设置为 0xFFFFFFFF
				若想关闭位置超限保护和超速保护，可以设置为 0xFFFFFFFC

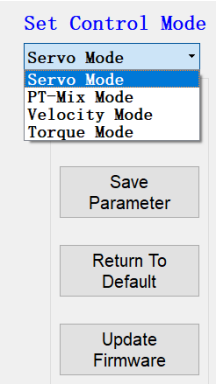
6.7.3 修改控制模式

您可以在 MParameter Studio 中修改控制模式，方法如下：

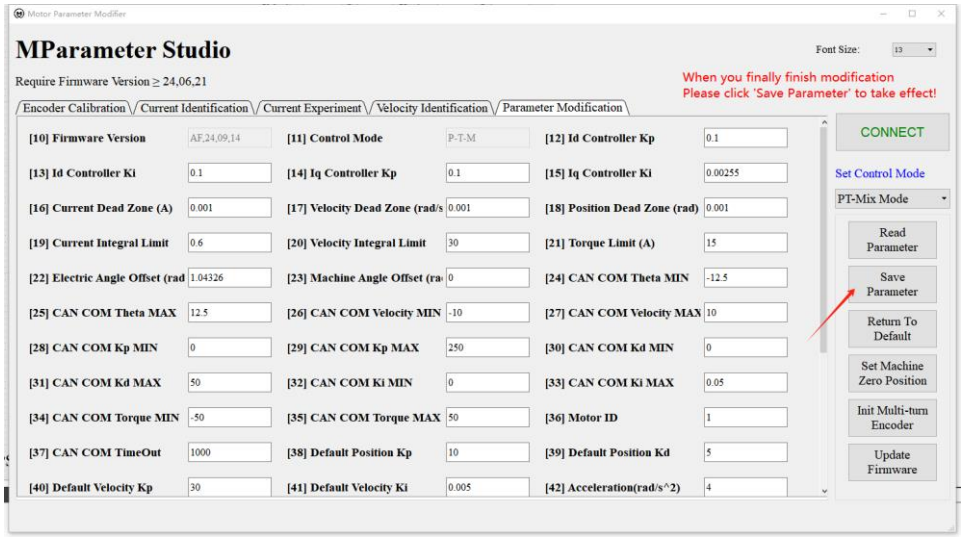
- Step 1
  按照 6.7.1 章节所示的方法建立成功建立连接
- Step 2
  查看当前控制模式，您可以在如图所示的红色框中看到当前的控制模式，并在蓝色框中选择期望的控制模式。



- Step 3
  选择您期望的控制模式



- Step 4
  单击'Save Parameter'按键，确保您的更改生效，修改控制模式随即完成。



6.7.4 固件升级

该功能尚待完善，请继续关注。

6.8 EtherCAT 控制

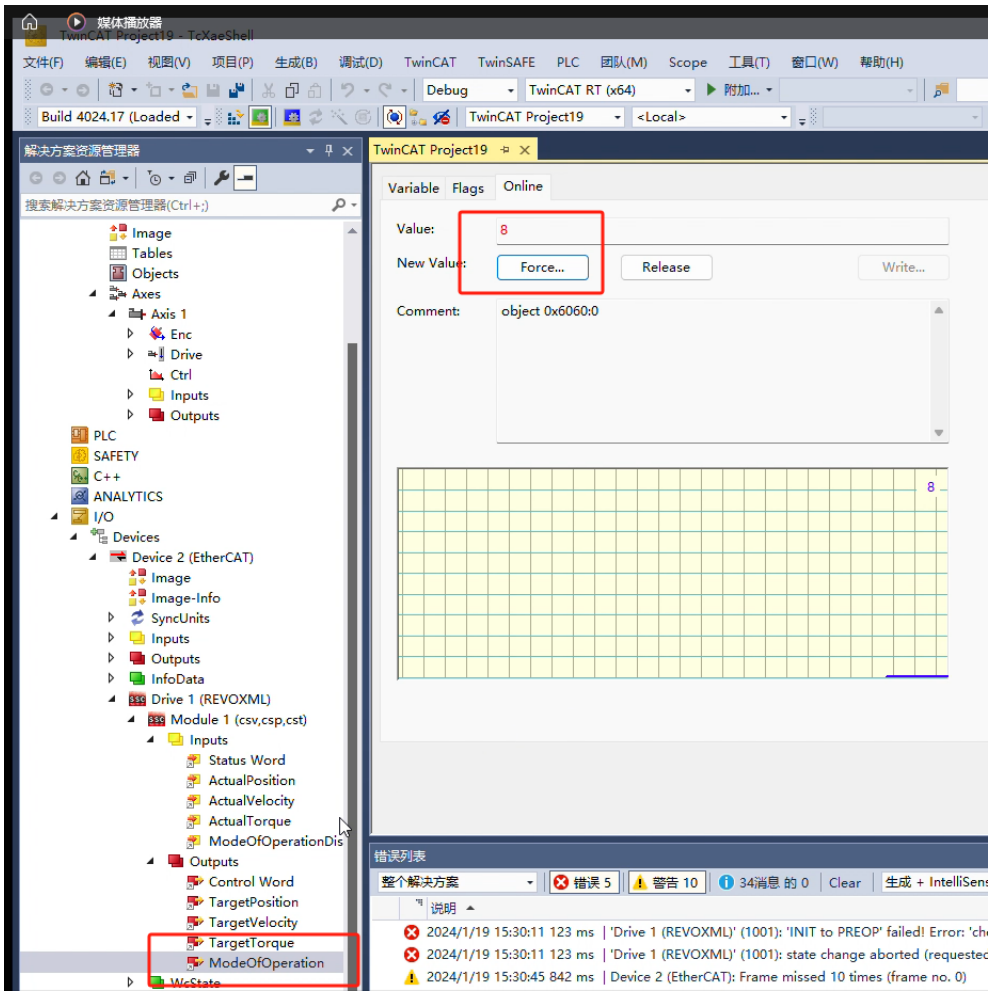
6.8.1 使用 EtherCAT 控制电机在 CSP 模式运行

当使用 EtherCAT 连接进行 CSP 控制时，请仔细查阅以下说明。

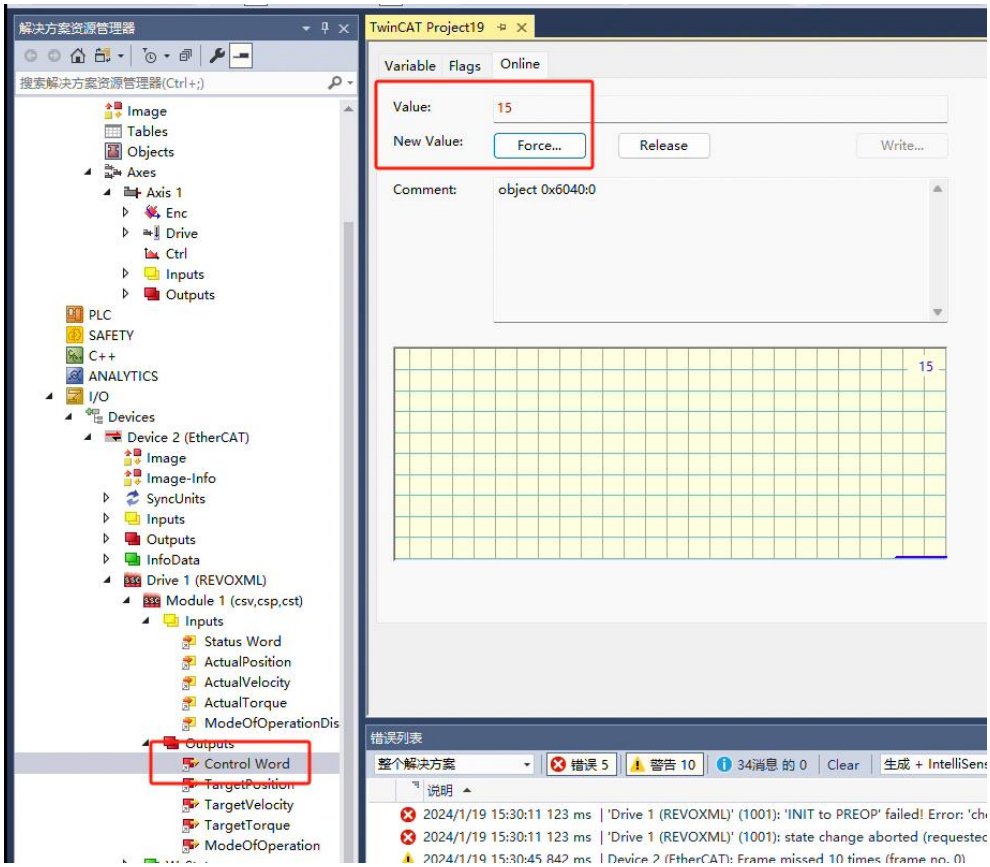
EtherCAT 连接进行 CSP 控制时，使用到位置环 PD 控制、速度环 PI 控制，其参数无法在线修改，需要插入 USB 线，使用 Motorevo Studio V3.2 及以上进行修改，具体可以参考 6.7 章。其中 *Default Position Kp* 表示位置环 Kp，*Default Position Kd* 表示位置环 Kd，*Default Velocity Kp* 表示速度环 Kp，*Default Velocity Ki* 表示速度环 Ki。您可以使用出厂时默认的参数，该参数调教的较为保守，您也可以手动整定该参数。关于参数的整定，推荐先使用 CAN 通讯调试出一个满足运动控制需求的参数，随后固定。

在实际使用时，请按照以下步骤进行：

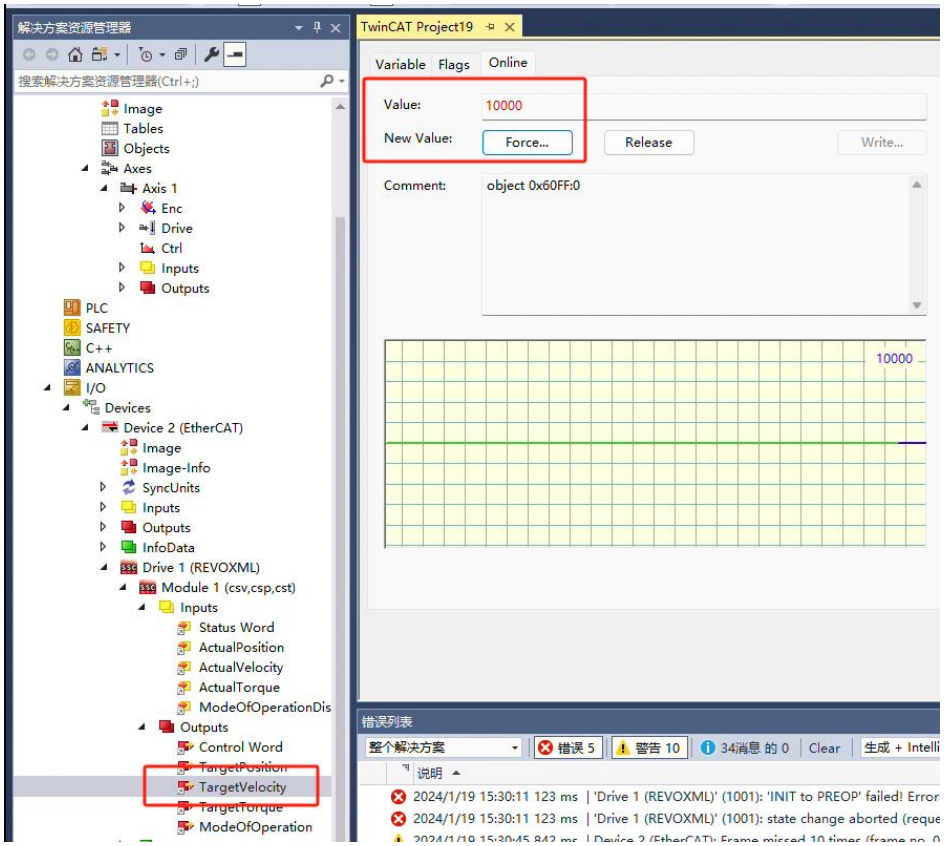
- Step 1：连接电源线、EtherCAT 控制线，确保驱动器表面无异物。
- Step 2：上电后，确保电机自检通过
- Step 3：设置 **ModeOfOperation** 的值为 **8（0x08）**，这将使电机进入 CSP 模式，对应 6.2.1 章中的伺服模式。



- Step 4：设置 **Control Word** 的值为 **15（0x0F）**，这将使电机进入 6.1 章中描述的 **Motor State** 状态，此时若驱动器配备了电磁式抱闸刹车，刹车将释放，电机进入运转状态。

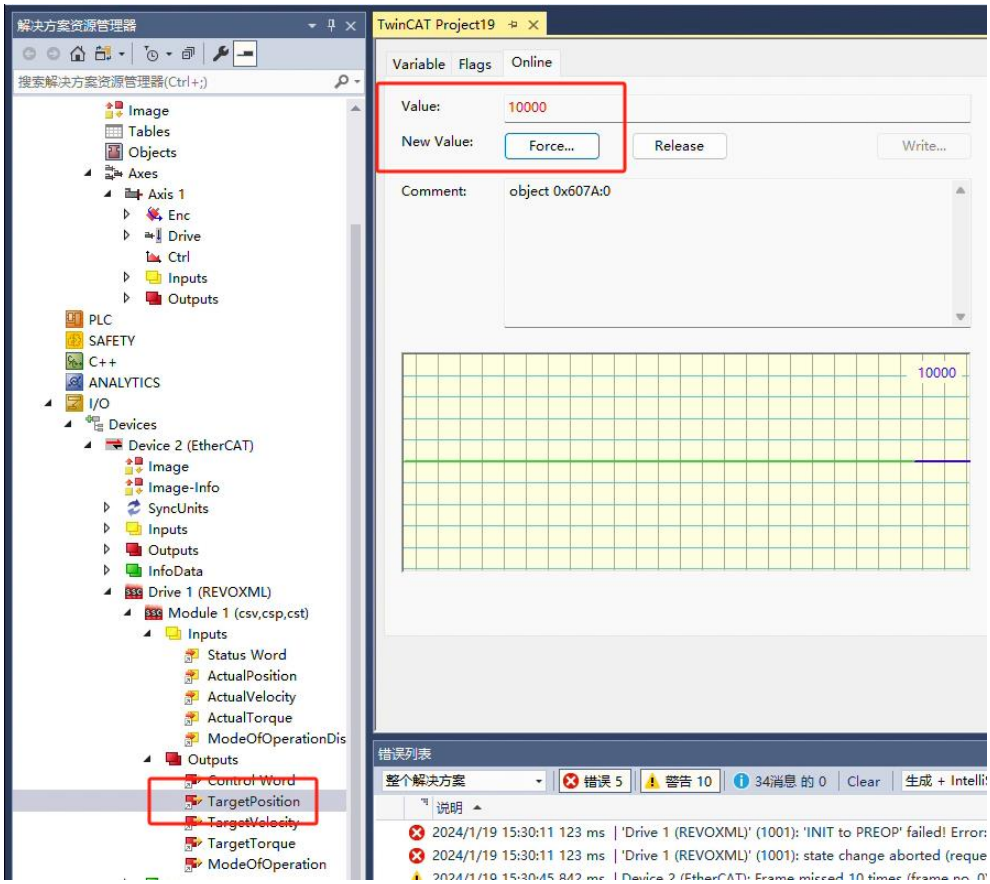


- Step 5：设置 **TargetVelocity**，这将决定电机向目标位置运行时速度，该变量的单位为 0.0001rad/s。例如，我们希望电机以 1rad/s 的转速向目标位置运行，则设置 **TargetVelocity** 的值为 **10000（0x2710）**。



Copyright ©2021-2022 Motorevo. All right reserved.
http://www.motorevo.cn

- Step6：设置目标位置 **TargetPosition**，电机将会运动到该角度，该变量的单位为 0.0001rad。例如，我们希望电机运行到 1rad 的角度，则设置 **TargetPosition** 的值为 **10000（0x2710）**。



此外，当使用结束、或不需要电机运行时，建议将 **Control Word** 的值设为 **7（0x07）**，这将使电机进入 6.1 章中描述的 **Rest State** 状态，此时若驱动器配备了电磁式抱闸刹车，刹车将吸合，确保电机不会转动。

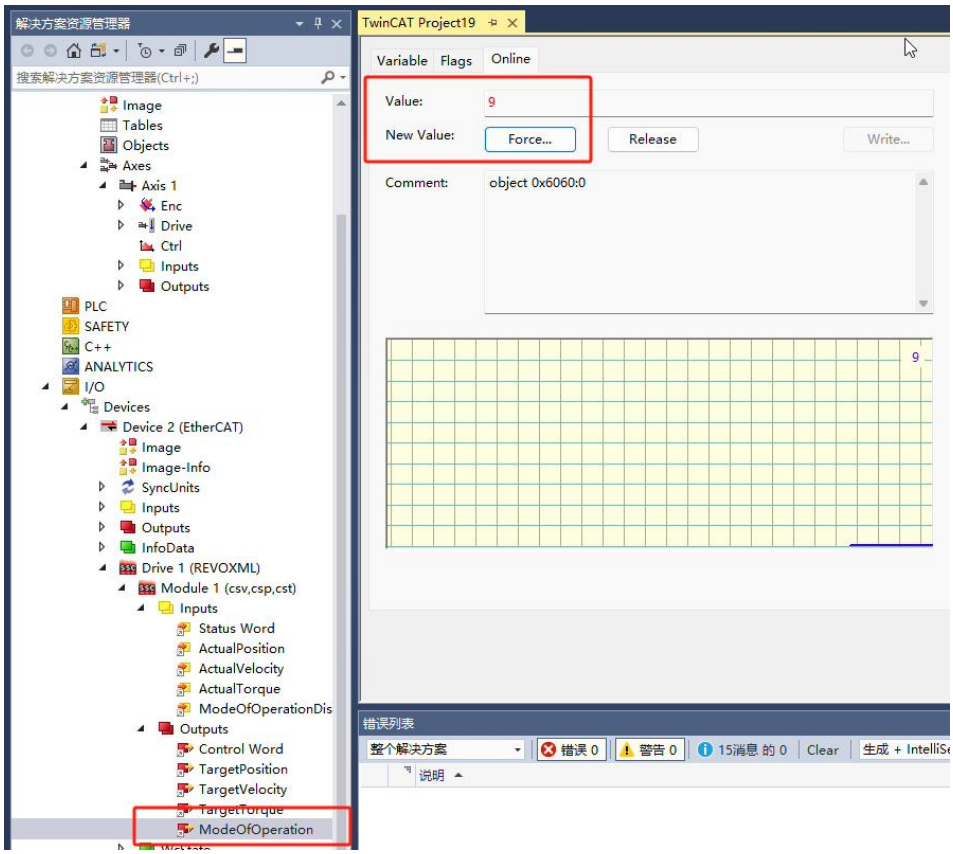
6.8.2 使用 EtherCAT 控制电机在 CSV 模式运行

当使用 EtherCAT 连接进行 CSV 控制时，请仔细查阅以下说明。

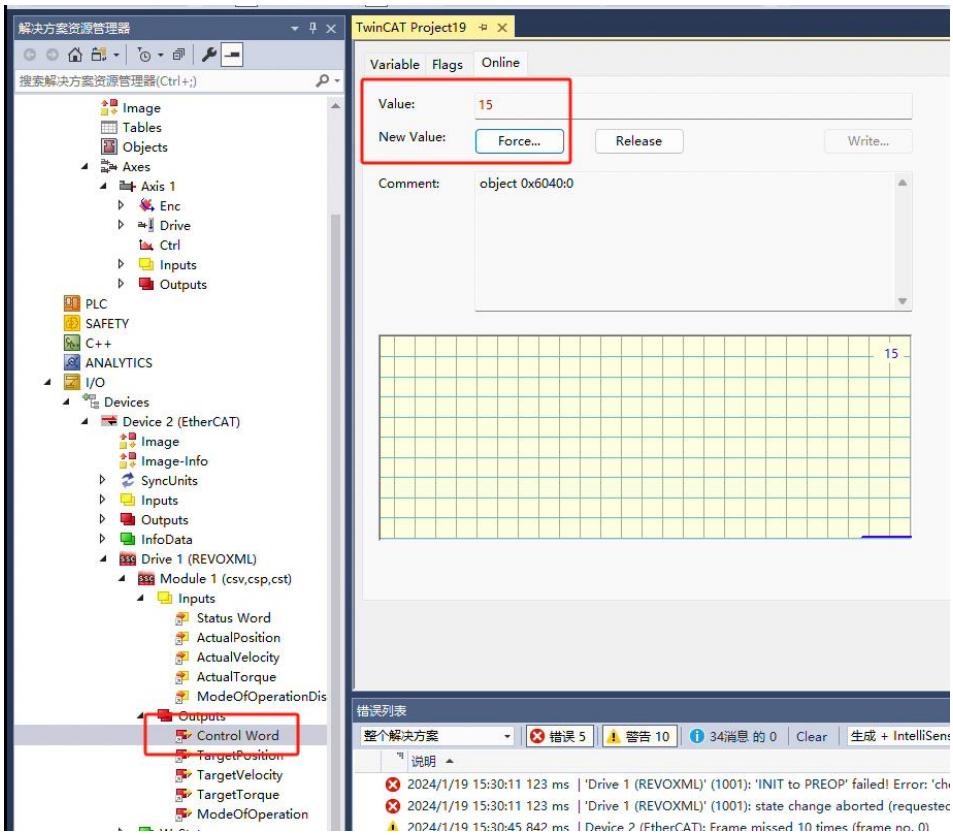
EtherCAT 连接进行 CSV 控制时，使用速度环 PI 控制，其 PID 参数无法在线修改，需要插入 USB 线，使用 Motorevo Studio V3.2 及以上进行修改，具体可以参考 6.7 章。其中 *Default Velocity Kp* 表示速度环 Kp，*Default Velocity Ki* 表示速度环 Ki。您可以使用出厂时默认的参数，该参数调教的较为保守，您也可以手动整定该参数。关于参数的整定，推荐先使用 CAN 通讯调试出一个满足运动控制需求的参数，随后固定。

在实际使用时，请按照以下步骤进行：

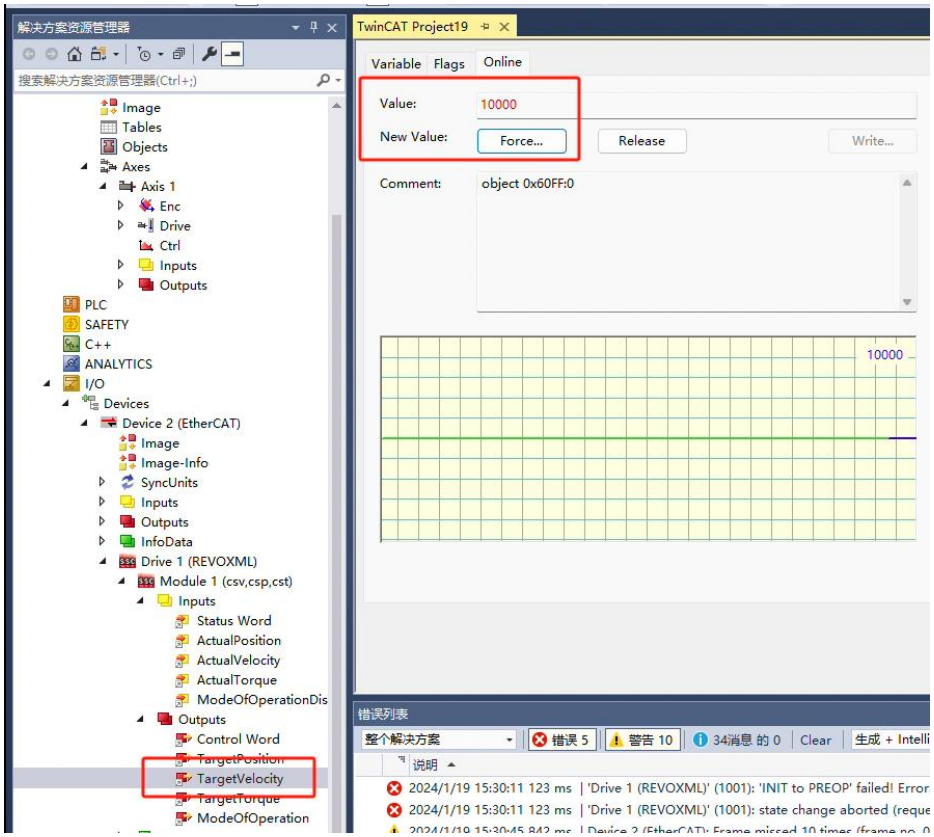
- Step 1：连接电源线、EtherCAT 控制线，确保驱动器表面无异物。
- Step 2：上电后，确保电机自检通过
- Step 3：设置 **ModeOfOperation** 的值为 **9（0x09）**，这将使电机进入 CSV 模式，对应 6.2.3 章中的速度模式。



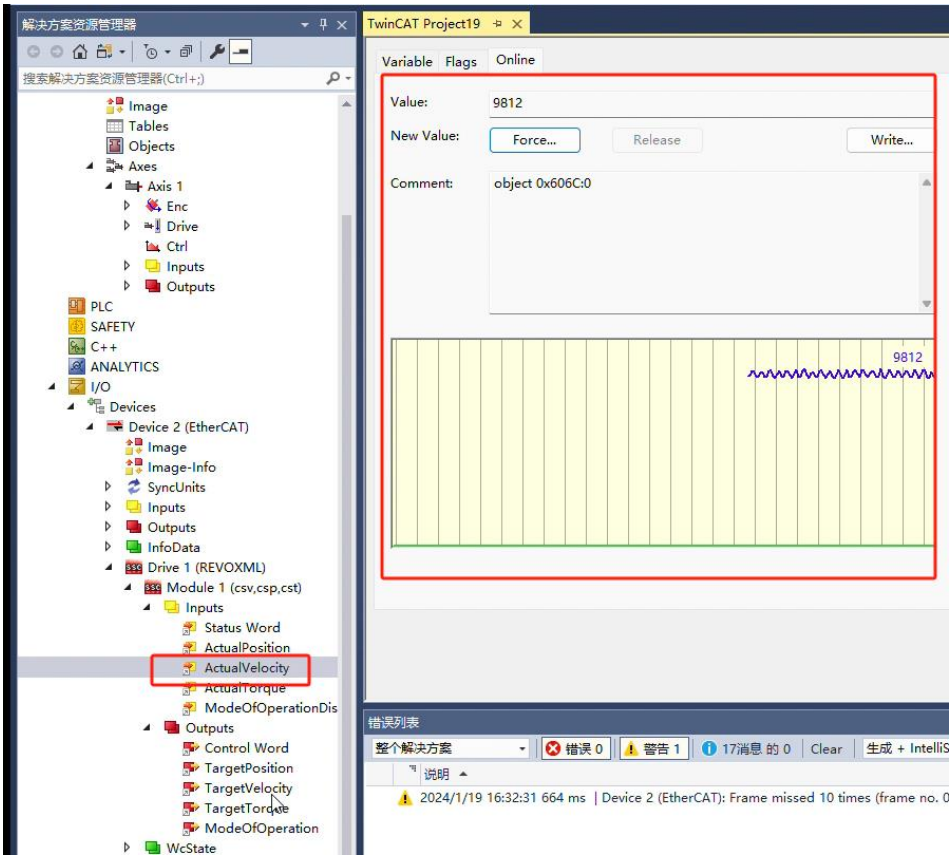
- Step 4: 设置 **Control Word** 的值为 **15（0x0F）**，这将使电机进入 6.1 章中描述的 **Motor State** 状态，此时若驱动器配备了电磁式抱闸刹车，刹车将释放，电机进入运转状态。



- Step 5: 设置 **TargetVelocity**, 这将决定电机的运行速度, 该变量的单位为 0.0001rad/s。例如，我们希望电机以 1rad/s 的转速运行，则设置 **TargetVelocity** 的值为 **10000（0x2710）**。



此时，可以在 **ActualVelocity** 中看到实际速度，单位也为 0.0001rad/s。



此外，当使用结束、或不需要电机运行时，建议将 **Control Word** 的值设为 **7（0x07）**，这将使电机进入 6.1 章中描述的 **Rest State** 状态，此时若驱动器配备了电磁式抱闸刹车，刹车将吸合，确保电机不会转动。

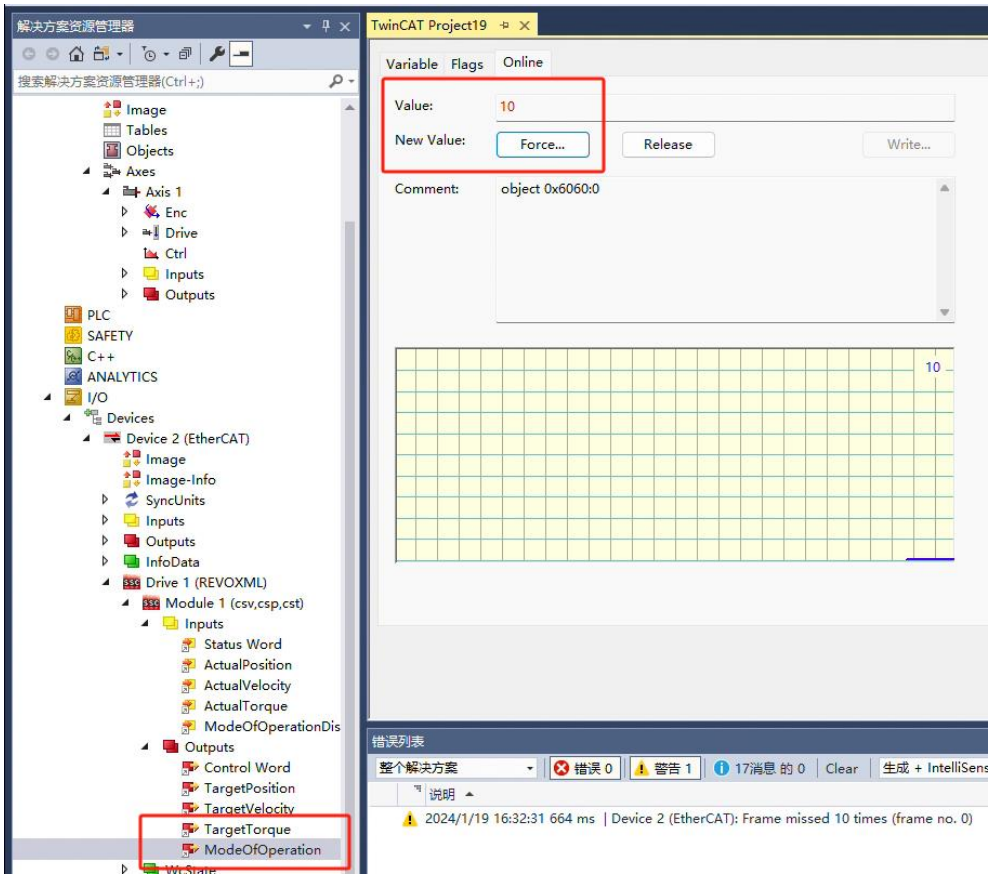
6.8.3 使用 EtherCAT 控制电机在 CST 模式运行

当使用 EtherCAT 连接进行 CST 控制时，请仔细查阅以下说明。

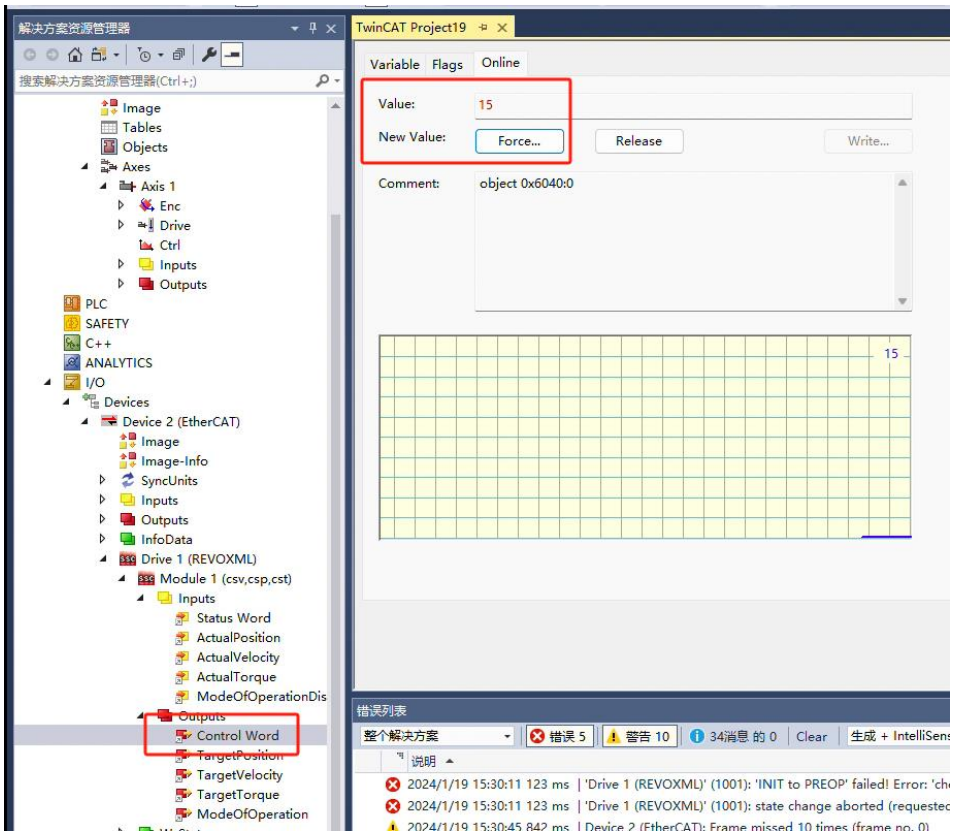
EtherCAT 连接进行 CST 控制时，使用电流环参数，其参数无法在线修改，需要插入 USB 线，使用 Motorevo Studio V3.2 及以上进行修改，具体可以参考 6.7 章。您可以使用出厂时默认的参数，该参数调教的较为保守，您也可以手动整定该参数。关于参数的整定，推荐先使用 CAN 通讯调试出一个满足运动控制需求的参数，随后固定。

在实际使用时，请按照以下步骤进行：

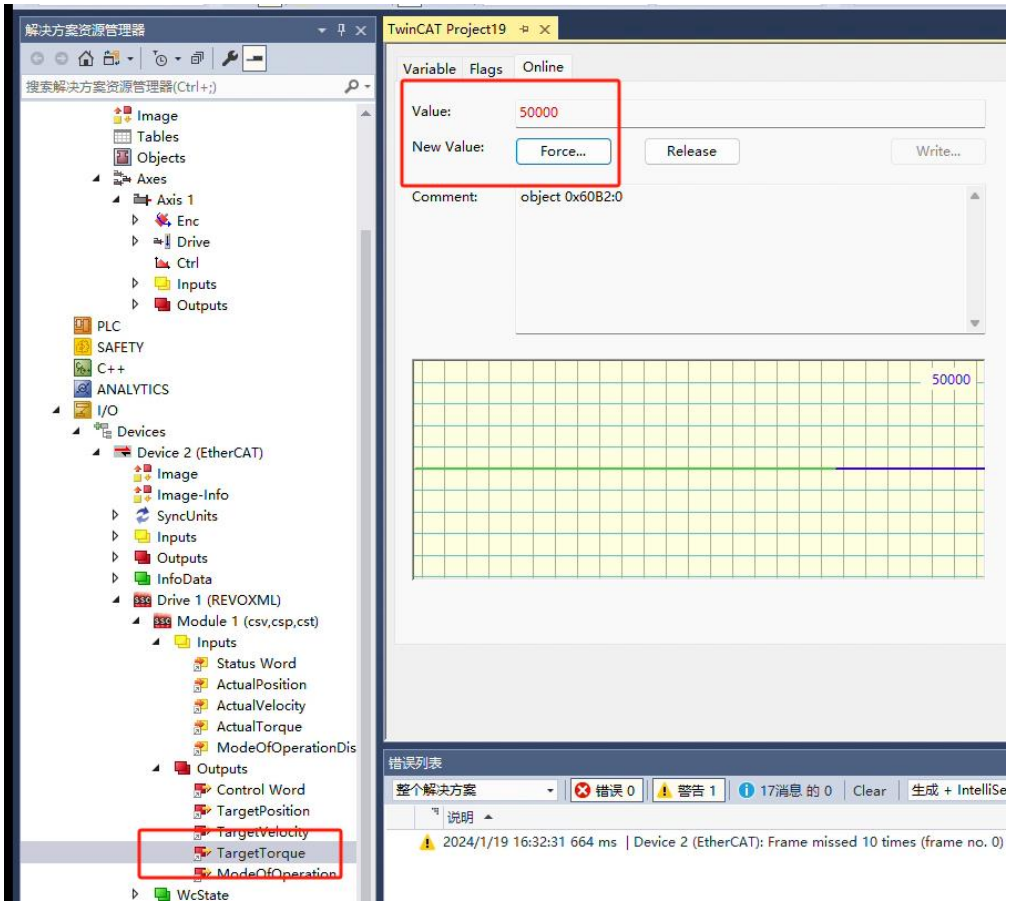
- Step 1：连接电源线、EtherCAT 控制线，确保驱动器表面无异物。
- Step 2：上电后，确保电机自检通过
- Step 3：设置 **ModeOfOperation** 的值为 **10（0x0A）**，这将使电机进入 CST 模式，对应 6.2.4 章中的力矩模式。



- Step 4: 设置 **Control Word** 的值为 **15（0x0F）**，这将使电机进入 6.1 章中描述的 **Motor State** 状态，此时若驱动器配备了电磁式抱闸刹车，刹车将释放，电机进入运转状态。



- Step 5：设置 **TargetTorque**，这将决定电机的输出扭矩，该变量的单位为 0.0001Nm。例如，我们希望电机输出扭矩为 5Nm，则设置 **TargetTorque** 的值为 **50000（0xC350）**。



此外，当使用结束、或不需要电机运行时，建议将 **Control Word** 的值设为 **7（0x07）**，这将使电机进入 6.1 章中描述的 **Rest State** 状态，此时若驱动器配备了电磁式抱闸刹车，刹车将吸合，确保电机不会转动。

6.9 通过 CAN 更改电机 Flash 中的参数

在 2024.9.25 后的新固件中，6.7 章中的参数可以通过 CAN 协议进行修改和保存，协议如下：

CAN 主站到电机的协议定义为：
CAN 报文类型：标准帧
CAN 报文标识符：0x600+Motor_ID

Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7
0x67	Index	Data0	Data1	Data2	Data3	Read/Write	0x76

参数解释：
Byte0：帧头，固定为 0x67。
Byte1：索引，确定电机读取修改对象，索引和参数的对应关系可见 6.7.2 表中查询。
Byte2：数据第 1 位。
Byte3：数据第 2 位。
Byte4：数据第 3 位。
Byte5：数据第 4 位。
Byte6：读写控制位，读命令位 0x04（读命令时 Byte2-Byte5 无要求），写命令为 0x15。
Byte7：帧尾，固定为 0x76。
＊除了更改参数，该协议还配套了三种操作命令：保存 Flash，回退默认参数以及校准电角度，此时 Byte2~Byte6 无要求，全发送 0x00 即可。
Save Flash: Index = 0x00
Return Default Parameter: Index = 0x02
Calibrate Electric Angle Offset: Index = 0x03

电机反馈给 CAN 主站的帧协议定义为：
CAN 报文类型：标准帧
CAN 报文标识符：0x600+Motor_ID

Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7
Motor_ID	Index	Data0	Data1	Data2	Data3	0x00	0xFF

参数解释：
Byte0：Motor_ID。
Byte1：索引，确定电机读取修改对象，索引和参数的对应关系可见 6.7.2 表中查询。
Byte2：数据第 1 位。
Byte3：数据第 2 位。
Byte4：数据第 3 位。
Byte5：数据第 4 位。
Byte6：固定为 0x00
Byte7：固定为 0xFF

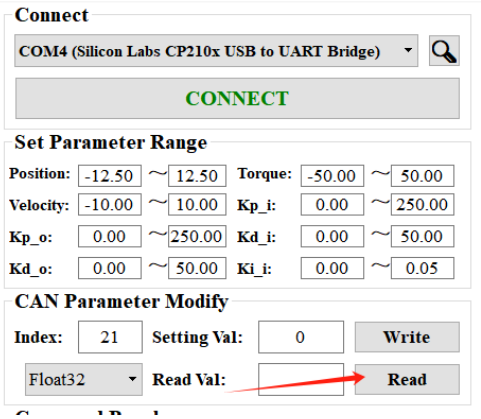
6.9.1 参数读写实例：

本章演示读写参数 Torque Limit（Index=21，数据类型为 Float）为例：
查询 Motor ID=1 电机的 Torque Limit 当前值，Torque Limit 的索引值为 21（十进制），转换为十六进制为 0x21：
发送数据：0x67 0x15 0x00 0x00 0x00 0x00 0x04 0x76，帧 ID 为 0x601
电机收到后反馈数据：0x01 0x15 0x00 0x00 0x70 0x41 0x00 0xFF，帧 ID 为 0x601
获得反馈数据为 0x41700000，根据 IEE754 标准解算得：15.0，现在将其改为 20.0（0x41A00000），则发送：0x67 0x15 0x00 0x00 0xA0 0x41 0x15 0x76，帧 ID 为 0x601
电机收到写命令后，修改完毕，并反馈最该参数修改后的值：
0x01 0x15 0x00 0x00 0xA0 0x41 0x00 0xFF，帧 ID 为 0x601，表明参数已经被修改成功。

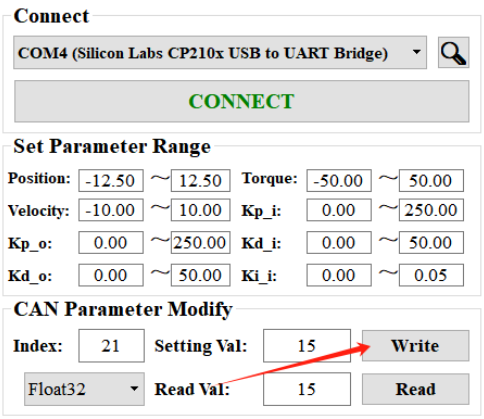
参数修改完毕后，可以发送 0x67 0x00 0x00 0x00 0x00 0x00 0x15 0x76，帧 ID 为 0x601 将修改结果保存于 Flash 中。

6.9.2 通过上位机修改

在建立连接后，在 CAN Parameter Range 面板设定要读取的参数的索引 Index，以及数据类型（如果数据类型错误会导致乱码和解算失败），随后点击 Read 按钮：



随后在 Setting Val 栏中输入新的值，点击 Write 按钮发送，电机会将更新后的参数更新在 Read Val 栏中。



7 通过 CAN 总线使用 Motorevo Studio™控制执行器运行实例

本章可以参考视频：【Motorevo 关节驱动器保姆教程】 【精准空降到 05:11】
https://www.bilibili.com/video/BV1Uh4y1C7Xw/?share_source=copy_web&vd_source=0be83eaef43126aba089ca5f7d82e5de&t=311

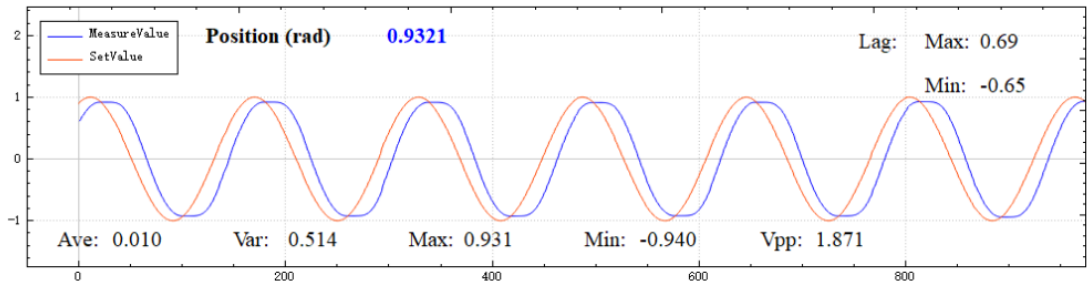
7.1 Motorevo Studio™界面

界面总览：

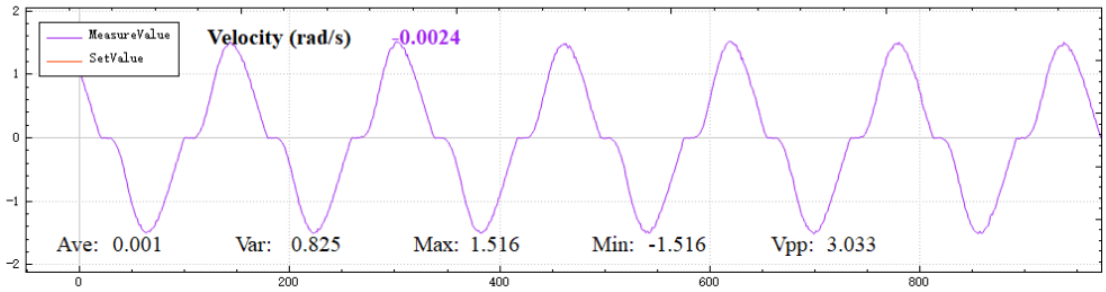


详情：

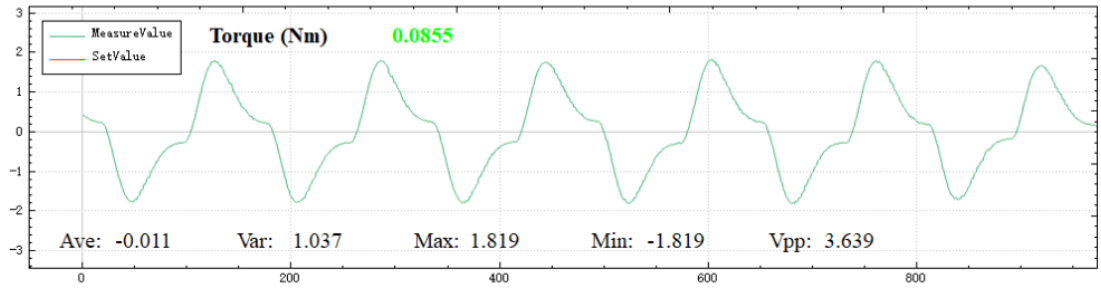
位置示波器：



速度示波器：



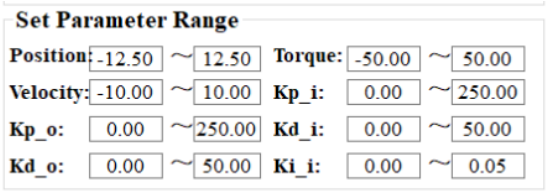
力矩示波器:



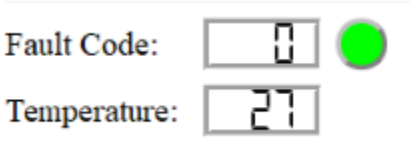
Jason 串口搜索连接面板:



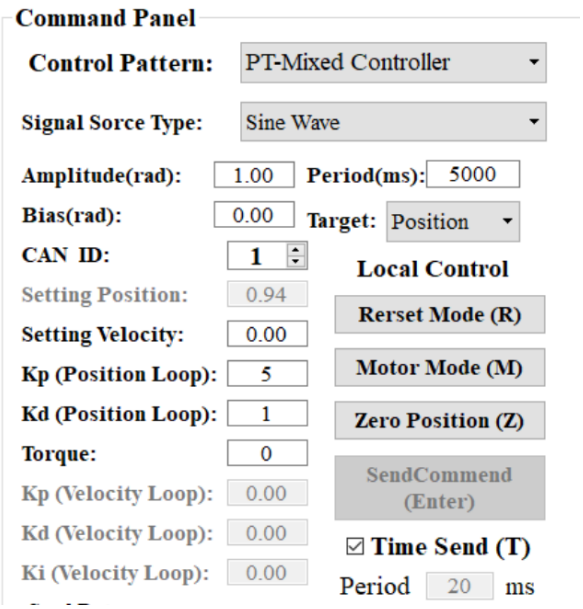
通信范围设置面板:



故障码以及温度显示面板:



运动控制面板:



发送数据监视面板:



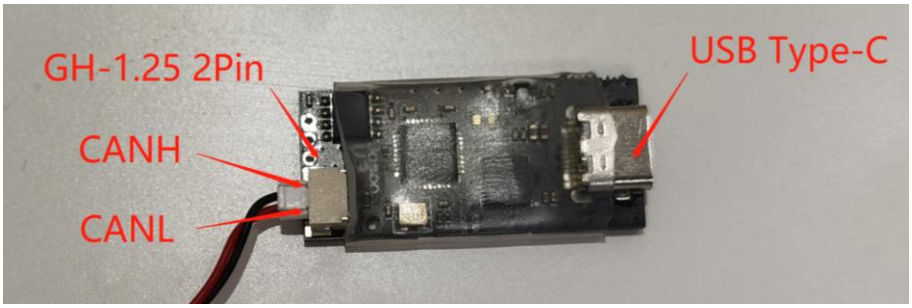
发包数、丢包数、丢包率监控:



7.2 通过 Jason 建立 PC-Actuator 连接

Jason 是一个特殊设计的模块, 可以将 PC 发出的 USB 信号转换为与执行器相匹配的 CAN 信号，实现 PC 与执行器之间的通信，只有借由 Jason，您才可以使用 Motorevo Studio™的 PC 上控制电机。

❗注意，在您使用 Jason 前，必须安装 CP2102 驱动，该驱动的固件可以在 https://github.com/SupremeLyy/Motorevo-Studio 上获取。

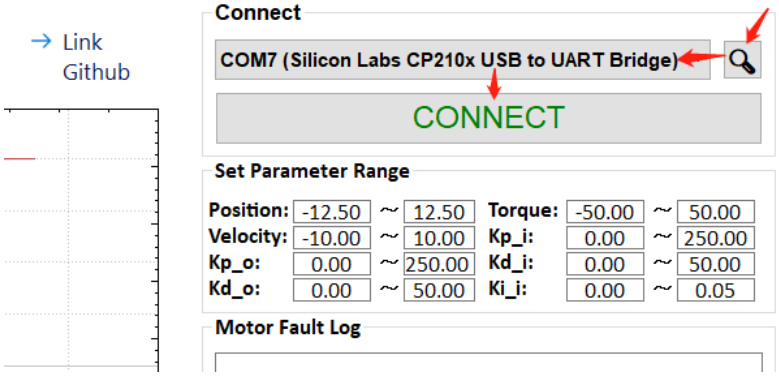


Jason Module

请准备一根 USB Type-C 数据线，将执行器和 PC 如左图连接，确保接线牢固后，通入主供电。

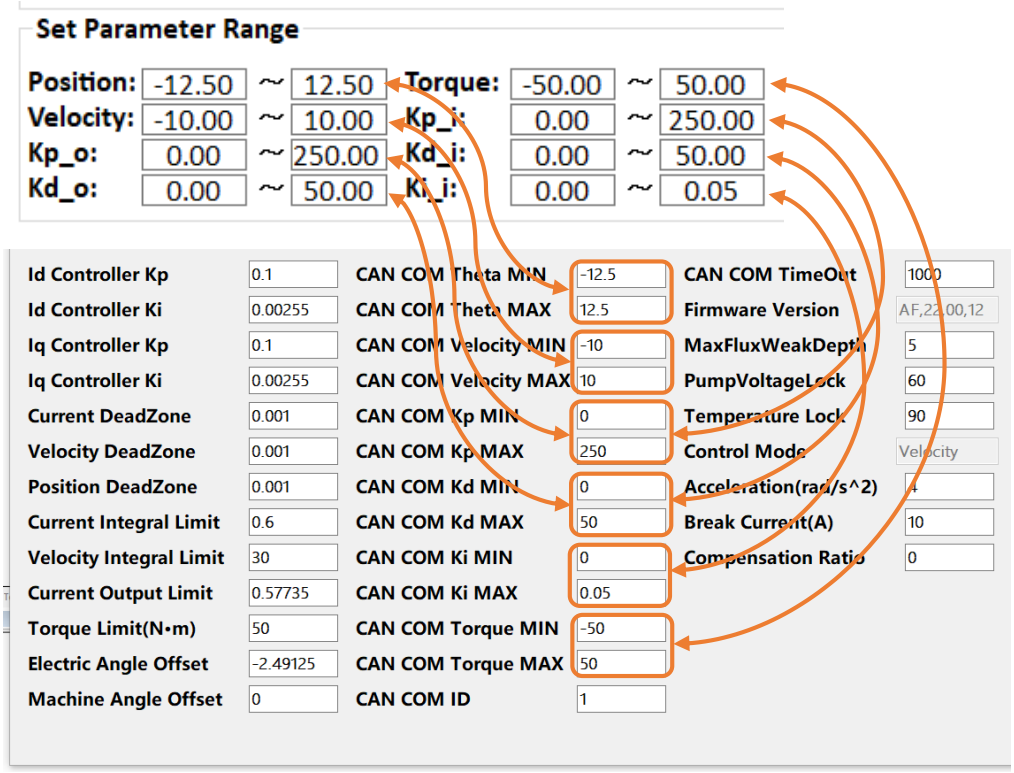


点击放大镜图标，在下拉菜单中选择 COMx (Silicon Labs CP210x USB to UART Bridge)，再点击 CONNECT 按键，若显示绿色 CONNECT 字样，则说明连接成功。若连接失败，请确认您的 PC 是否成功安装了 CP2102 驱动，并到您的设备管理器中确认是否识别到设备。

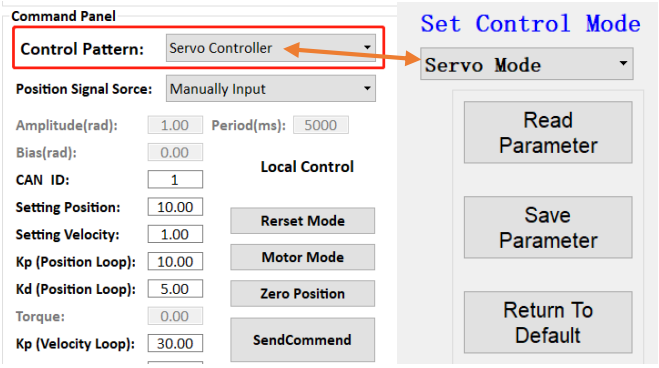


7.3 使用 Motorevo Studio™驱动电机运行

在开始运行之前，请确认 Motorevo Studio™中的范围参数和执行器 Flash 中的范围参数范围一致，若两边参数范围不一致，可能会造成严重的事故。

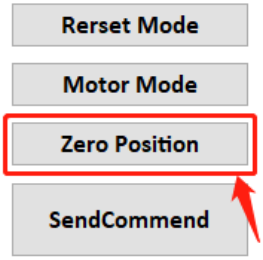


同时，确认 Motorevo Studio™中 Control Pattern 栏的控制模式和执行器的 Flash 中的一致。

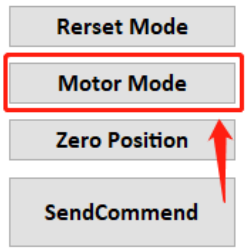


Motorevo Studio™ Actuator Flash

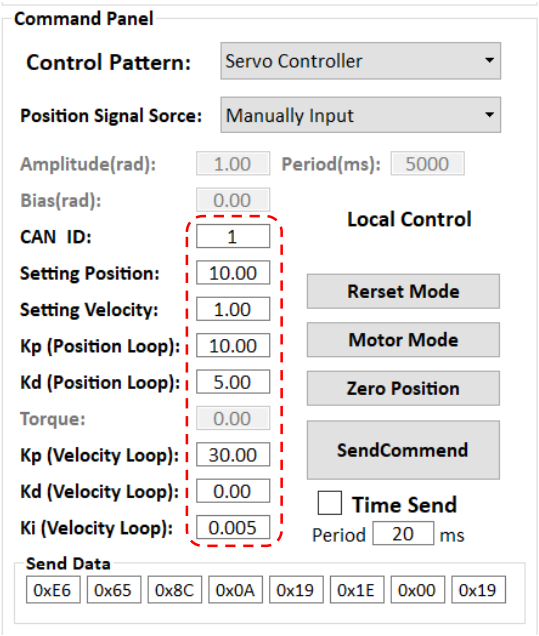
点击 Zero Position 按钮，设置当前角度为 0.0rad。



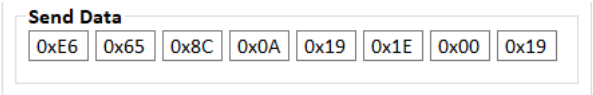
点击 Motor Mode 按钮，使得执行器进入 Motor State。



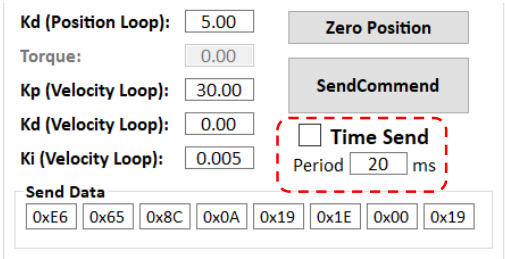
在 Servo 模式下，您可以在红框处输入 CAN ID，Target Position，Target Velocity，位置环、速度环参数，推荐的参数如下图：



Motorevo Studio™会自动将您输入的浮点类型指令转化为 8 个 8bits CAN 数据标准帧，您可以在 Send Data 栏中检视 CAN 数据。



勾选 Timer Send 框，使得 Motorevo Studio™按照设定的固定的周期发送参数，您可以在 Period 框中更改发送指令周期(Unit: ms)

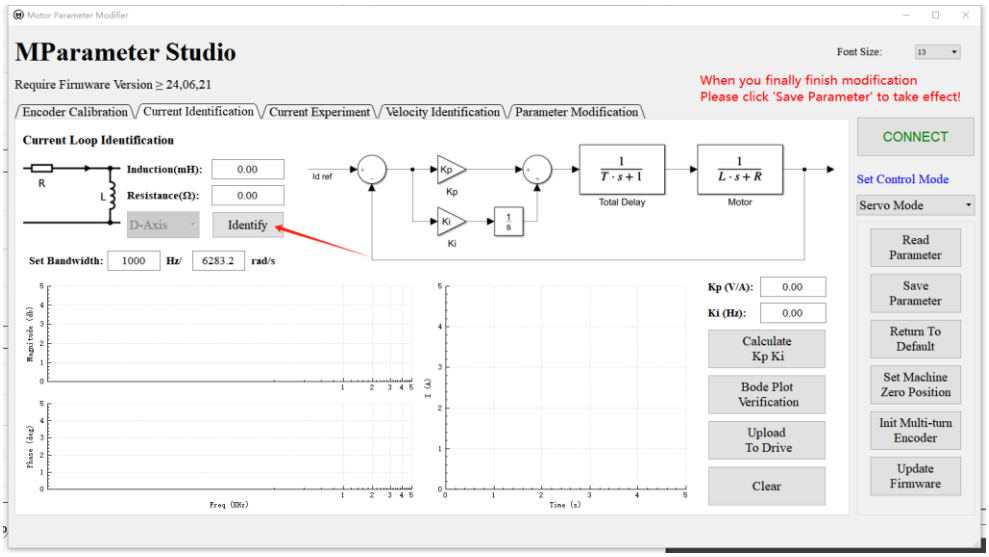


随后，您的执行器将开始转动，您可以在示波器中查看位置、速度、力矩曲线，或输入新的目标位置，观察执行器的运动情况。

7.4 电流环参数辨识以及电流环带宽测试

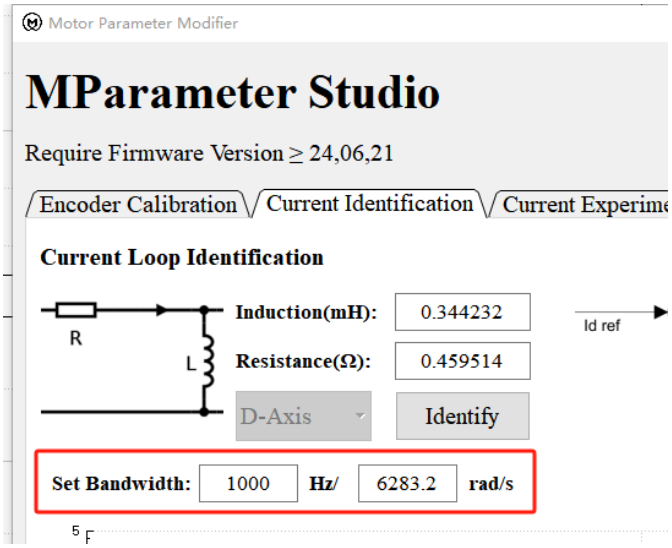
请按照如下步骤执行：

1. 给电机提供 24V 2A 供电，注意电源电压不可高于 28V，否则有烧坏电机的风险！
2. 插入 USB 线缆，同时断开 CAN 连接（或保持总线静默），随后建立连接。
3. 确保电机输出端无负载，防止电机因为转动引起的危险！
4. 在 Current Identification 界面，点击 Identify 按钮：

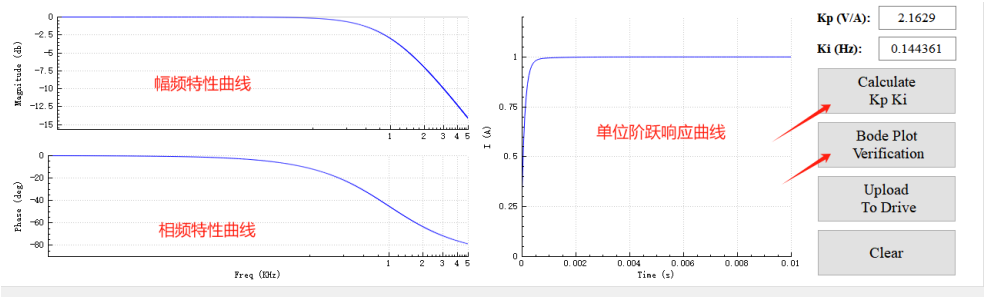


此时驱动板会自动电机 D 轴电感以及电阻，并显示。

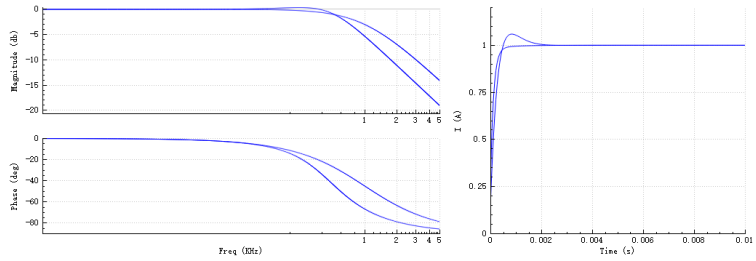
5. 输入期望电流环带宽：



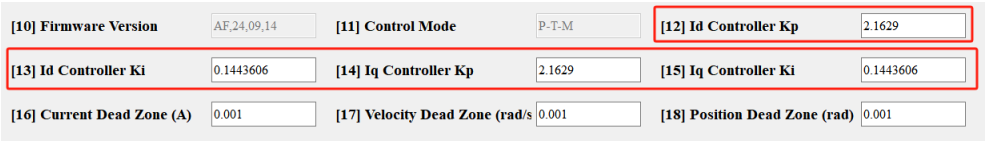
6. 点击 Calculate Kp Ki，计算 PI 参数，随后可以点击 Bode Plot 按钮绘制模型 Bode 图



您也可以设置不同的 PI 参数对比：

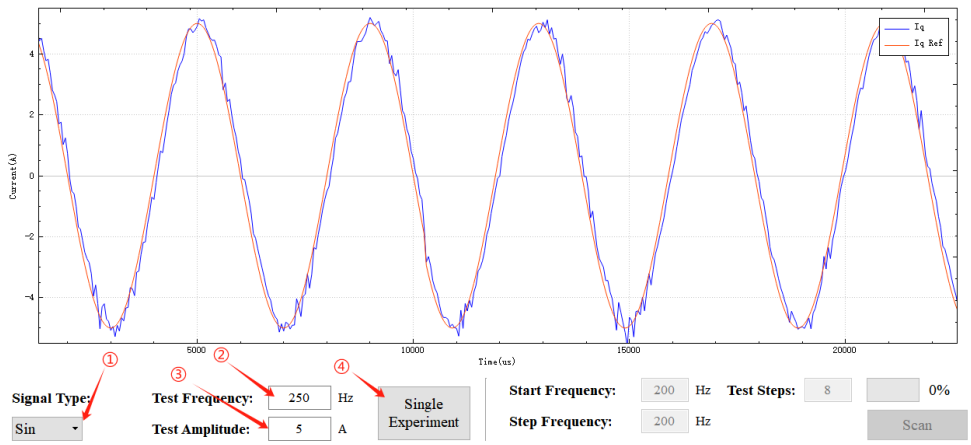


7. 点击 Upload To Drive，将 PI 参数发送至电机，注意点击完该按钮后需要回到 Parameter Modification 界面，将每个参数框下按下回车发送，最好在发送完毕后 Read 一遍检查。

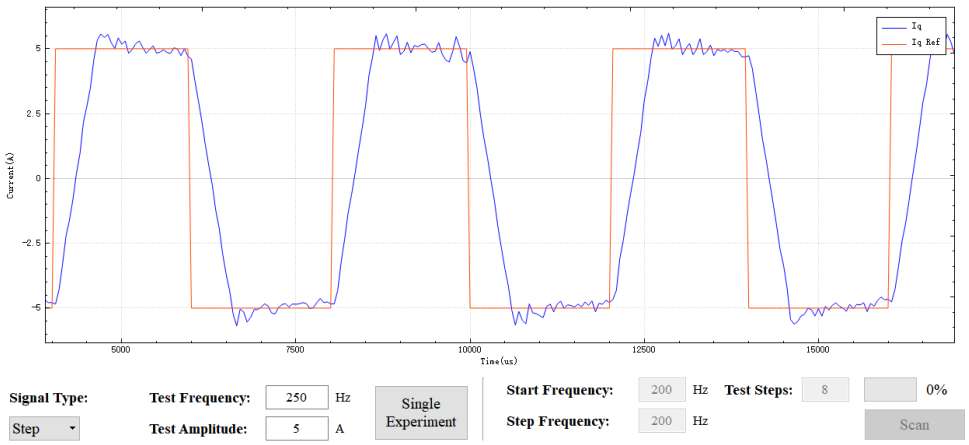


8. 如果要查看电流响应，请打开 Current Experiment 界面：先选择测试信号类型为正弦信号或方波阶跃信号，随后设置测试频率，测试幅值，最后点击 Single Experiment 按钮：

正弦信号测试（横轴为时间）：



方波信号测试（横轴为时间）：



7.5 位置环、速度环正弦、方波响应测试。

此功能已开发完毕，尚在内部测试中，如有需要请联系客服。

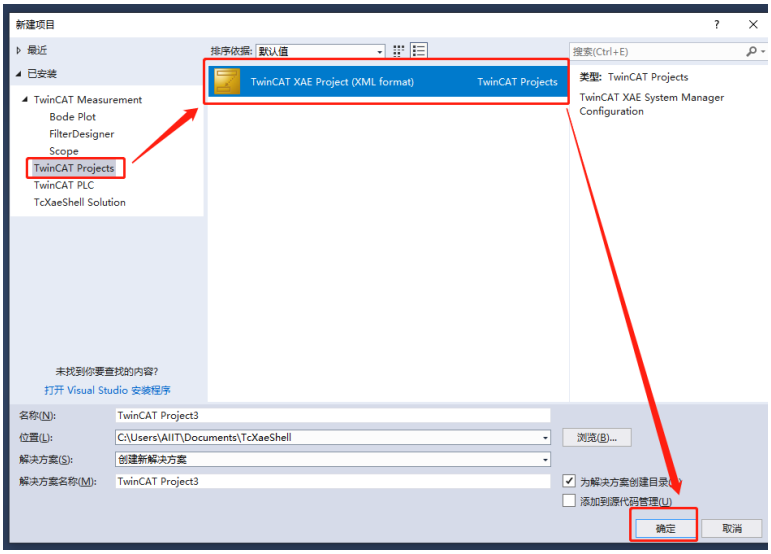
8 通过 EtherCAT 使用 TWINCAT3 和控制执行器运行实例

8.1 建立 ECAT 连接

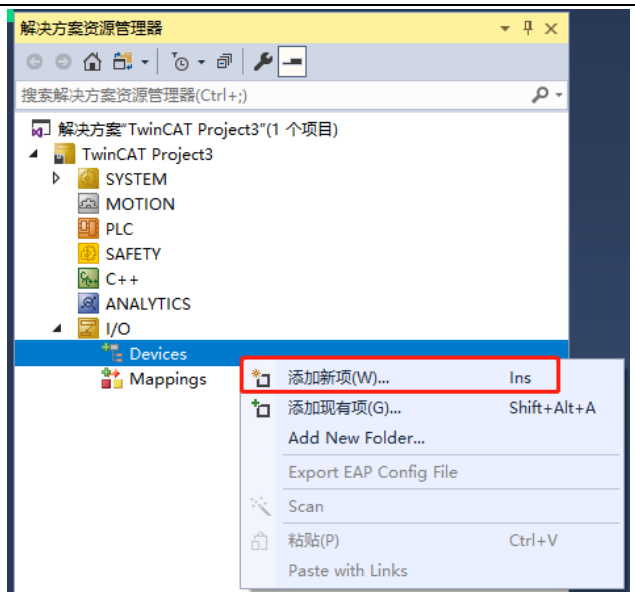
启动软件 TWINCAT3，新建项目。



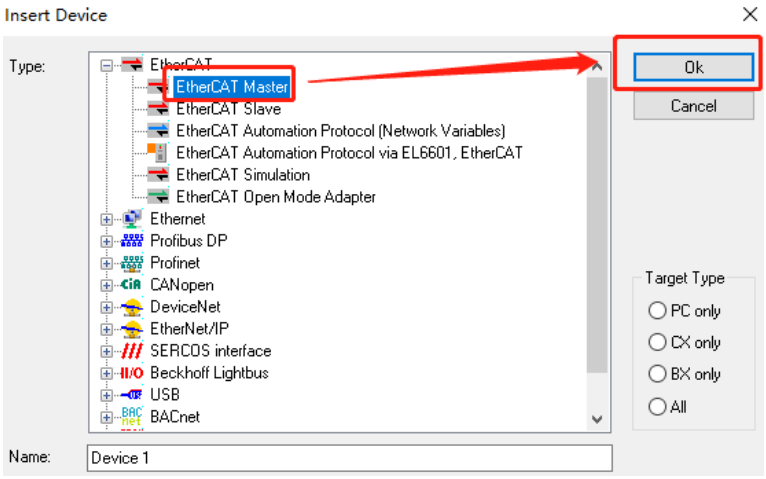
新建 TwinCAT Project，选择 TwinCAT XAE Project (XML format)，再点击确定按钮



下拉 I/O 菜单，在 Devices 选项上右键点击添加新项。



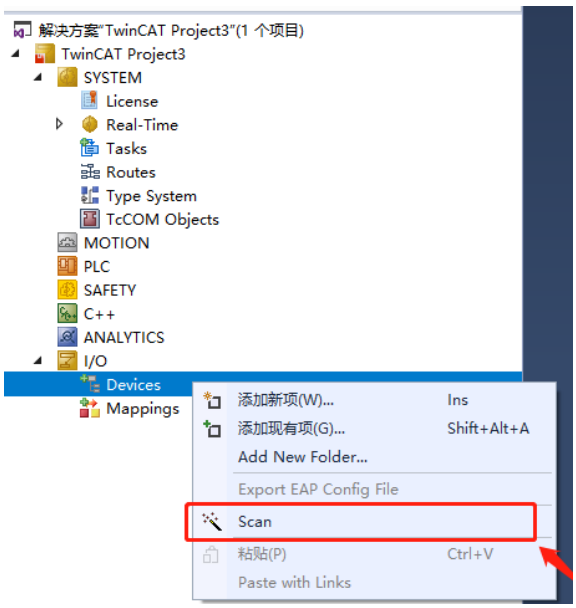
选择 EtherCAT Master 模式，再点击 OK 按钮。



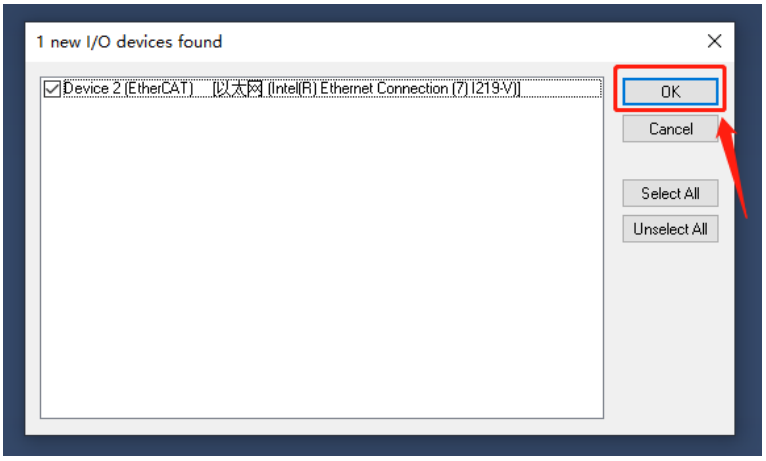
单击 Restart TwinCAT (Config Mode)。



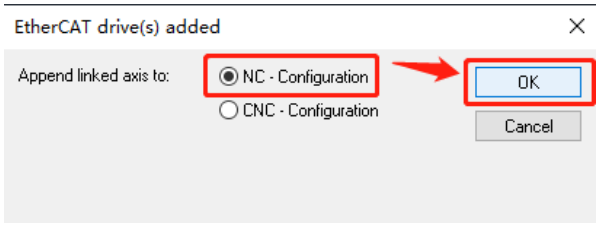
下拉 I/O 菜单，在 Devices 选项上右键点击 Scan.



若显示如下图所示，则说明 TwinCAT 扫描到 EtherCAT 协议设备，物理层链路连接正常，再点击 OK 按钮。



勾选 NC-Axis。

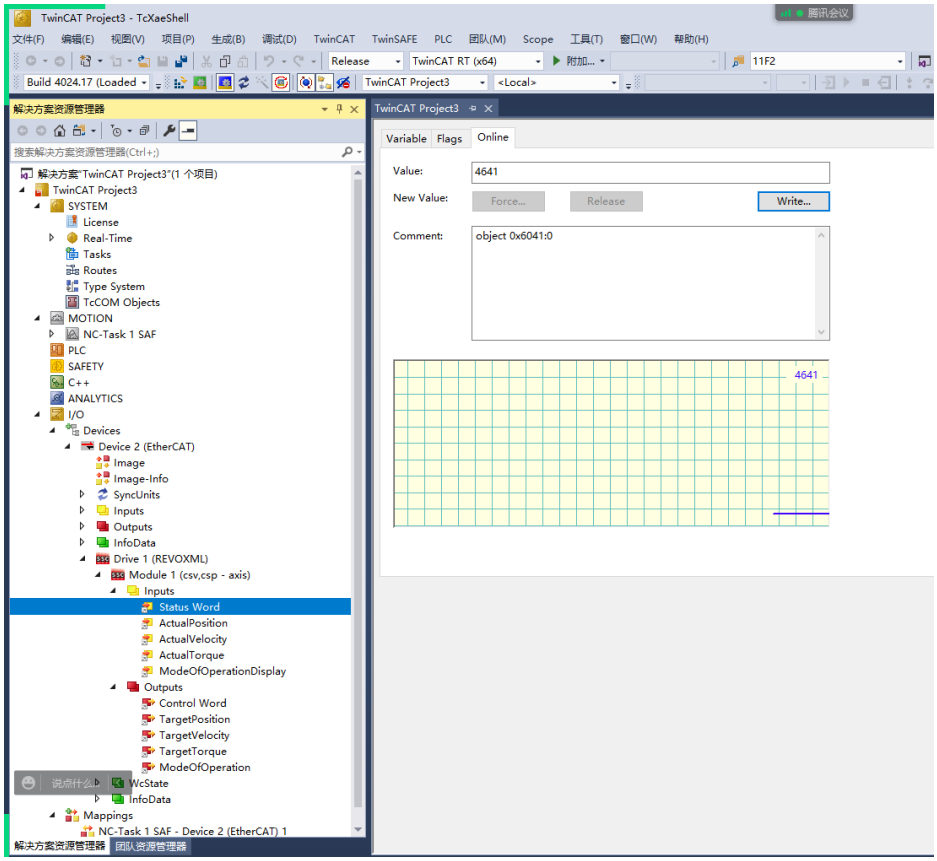


激活 Free-Run。



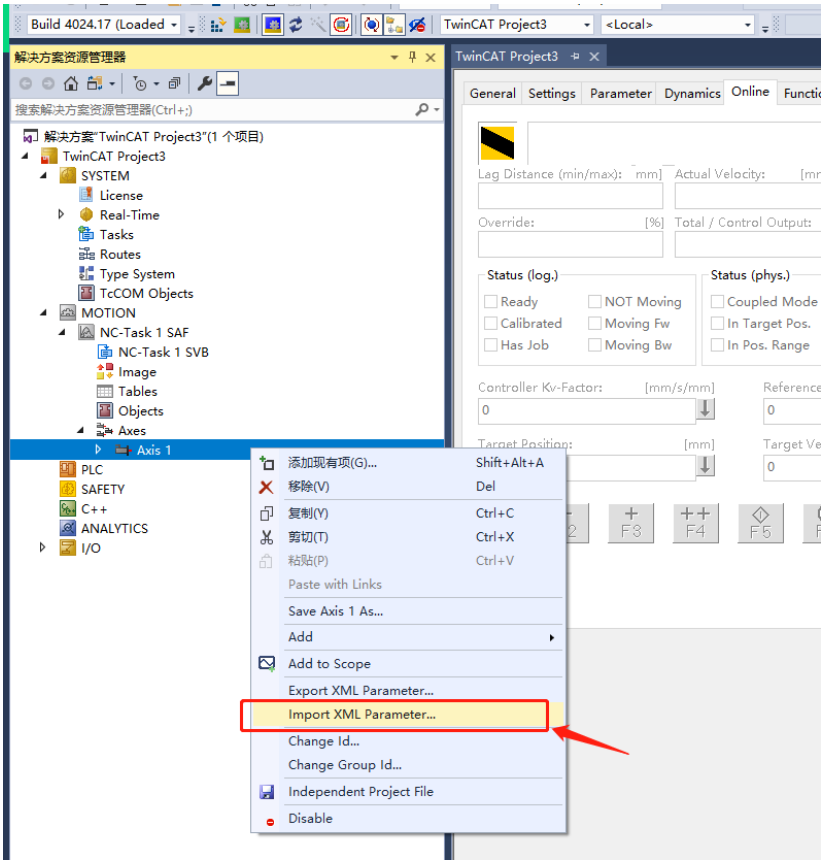
进入 I/O → Devices → Devices2(EtherCAT) → Drive1(REVOXML) → Module1(csv, csp-axis),

若显示如下图所示，说明 EtherCAT 通信正常。

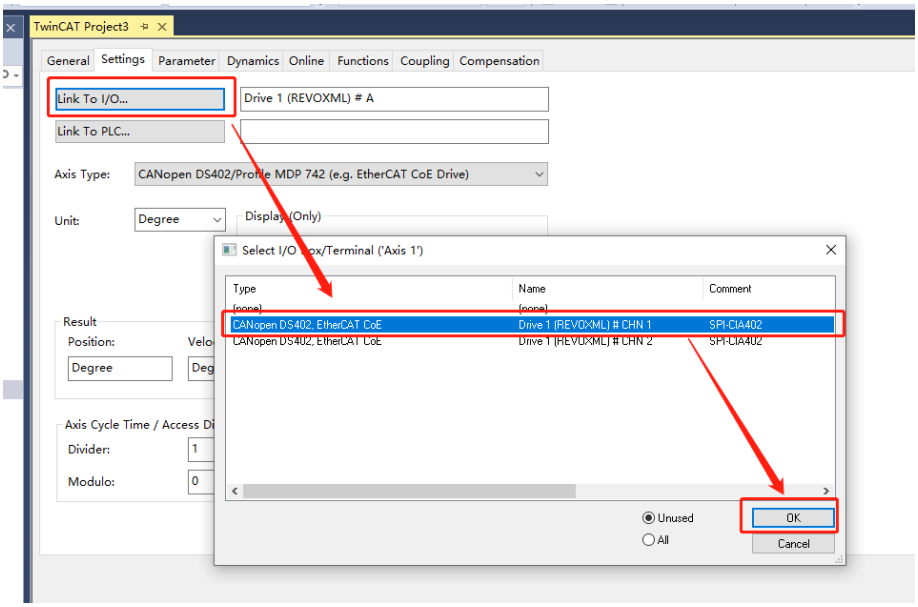


8.2 导入 XML 文件

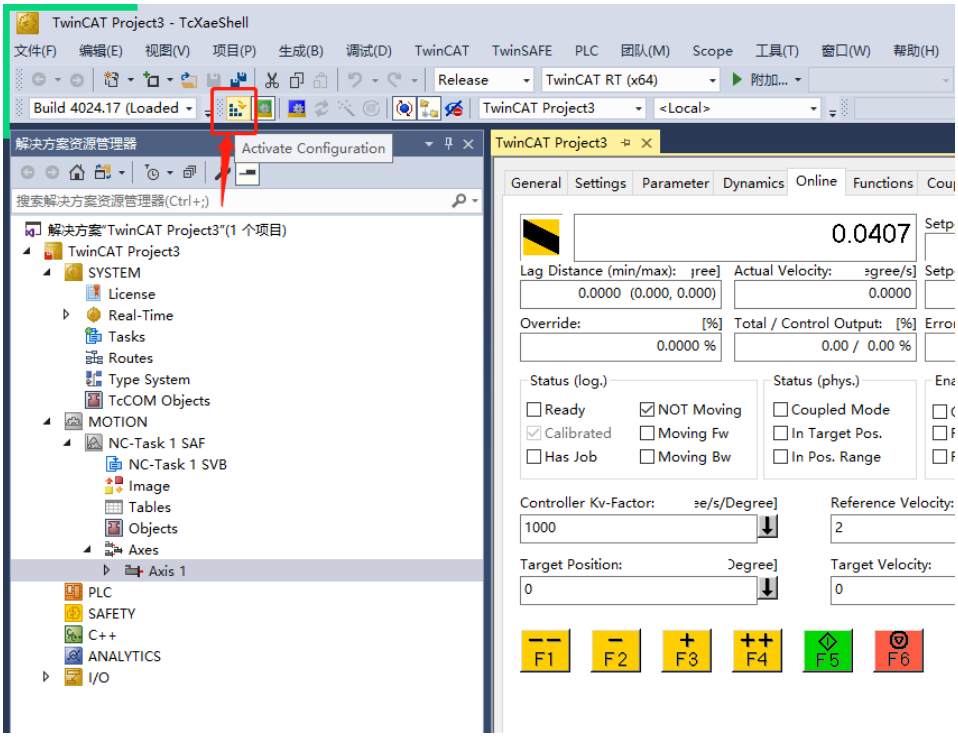
如图所指，在 Axis1 选项上右键点击 Import XML Parameter，导入提供的 NC_TOOL 文件



接下来，设置 Link To I/O 至#CHN1，再点击 OK 按钮。

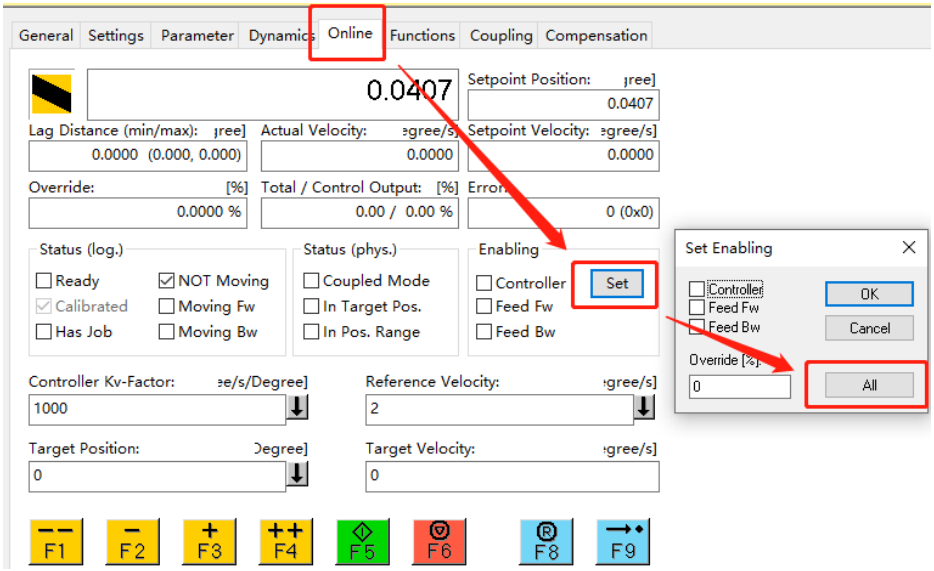


点击箭头所指处的 Activate Configuration 按钮，生效当前 XML 文件。

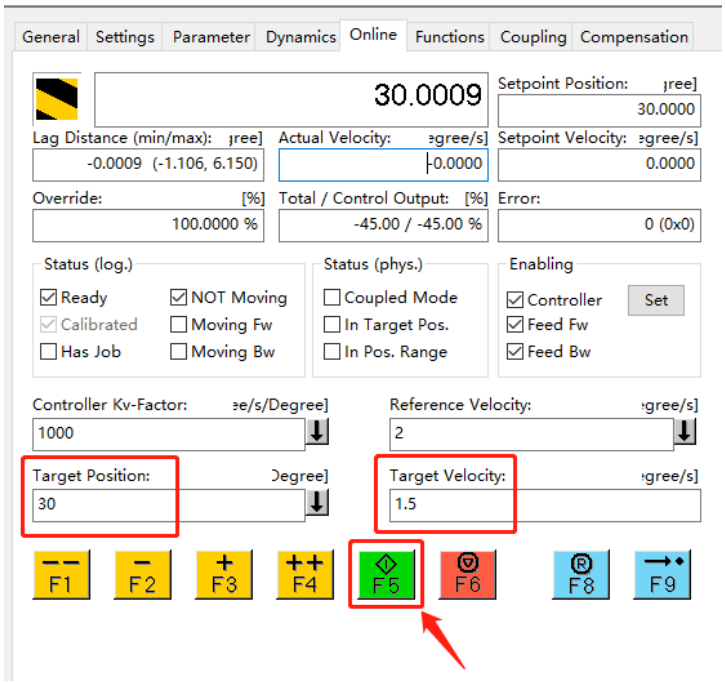

TwinCAT Project3 - TcXaeShell
文件(F) 编辑(E) 视图(V) 项目(P) 生成(B) 调试(D) TwinCAT TwinSAFE PLC 团队(M) Scope 工具(T) 窗口(W) 帮助(H)
Release
TwinCAT RT (x64)
Build 4024.17 (Loaded
TwinCAT Project3
<Local>
Activate Configuration
解决方案资源管理器
搜索解决方案资源管理器(Ctrl+;)
解决方案"TwinCAT Project3"(1 个项目)
TwinCAT Project3
SYSTEM
License
Real-Time
Tasks
Routes
Type System
TcCOM Objects
MOTION
NC-Task 1 SAF
NC-Task 1 SVB
Image
Tables
Objects
Axes
Axis 1
PLC
SAFETY
C++
ANALYTICS
I/O
TwinCAT Project3
General Settings Parameter Dynamics Online Functions
0.0407
Lag Distance (min/max):
0.0000 (0.000, 0.000)
Actual Velocity:
0.0000
Override: [%]
0.0000 %
Total / Control Output: [%]
0.00 / 0.00 %
Status (log.)
Ready
NOT Moving
Calibrated
Moving Fw
Has Job
Moving Bw
Status (phys.)
Coupled Mode
In Target Pos.
In Pos. Range
Controller Kv-Factor:
1000
Reference Velocity:
2
Target Position:
0
Target Velocity:
0
F1 F2 F3 F4 F5 F6

8.3 在 CSP 模式下运行

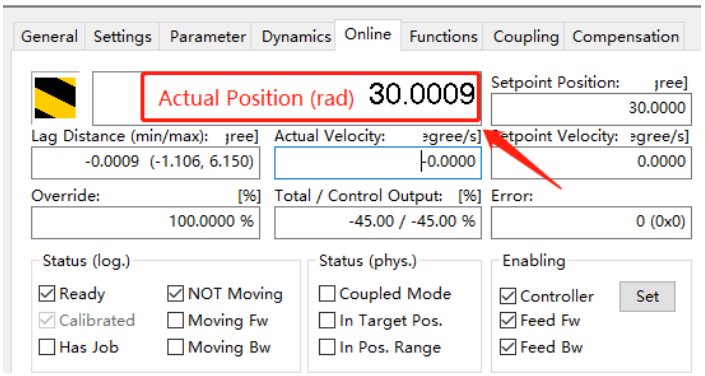
在 Online 模块下，点击 Set 按钮，在 Set Enabling 菜单中点击 All 按钮。



接下来，您可以在 Target Position 框中输入设定目标角度（单位：rad），在 Target Velocity 中设定运行速度（单位 rad/s），设定完成后，点击绿色的 F5 按钮，执行器随即开始从当前角度运行至目标角度。



您可以在此处监控当前电机的实际角度（单位：rad）。


General
Settings
Parameter
Dynamics
Online
Functions
Coupling
Compensation
Actual Position (rad) 30.0009
Setpoint Position:
30.0000
Lag Distance (min/max):
-0.0009 (-1.106, 6.150)
Actual Velocity:
0.0000
Setpoint Velocity:
0.0000
Override:
100.0000 %
Total / Control Output: [%]
-45.00 / -45.00 %
Error:
0 (0x0)
Status (log.)
Ready
NOT Moving
Calibrated
Moving Fw
Has Job
Moving Bw
Status (phys.)
Coupled Mode
In Target Pos.
In Pos. Range
Enabling
Controller
Feed Fw
Feed Bw
Set